

LaMagnética

SEARCH ENGINE OPTIMIZATION STRATEGY

SUBMITTED TO:

IGNACIO ANGUITA

SUBMITTED BY:

KAYLA DE SILVA

HARRY JONES

TIM REDING

MATIJA ULIC

Contents

1. Executive Summary 4
2. Description of the company 5
3. Aim of the Project 6
4. Scope of the Project 7
5. State of the Art 9
5.1 Introduction 9
5.2 Modern SEO Methods 9
5.3 AI and Machine Learning in SEO 11
5.4 Tools and Technologies 14
5.5 Competitors and Market 14
5.6 Conclusion 15
6. Development or Body of the Report 16
6.1 Data Preprocessing 16
6.2 Transforming and Encoding 17
6.3 Model Training 18
6.4 Feature Importance 40
7. Key Findings and Strategy Recommendations 48
8. Future Work 50
8.1 Validate Current Assertions 50
8.2 Develop Neural Network 51
8.3 Performance Metric Analysis 52
8.4 Competitor Analysis 52
8.5 Scaling Framework 53
9. Conclusion 55
10. Bibliography / References 57
11. Technical Annex 59
11.1 Sentence Transformers 59
11.2 Evaluation Function 61
11.3 Custom Loss Function for XGBoost 65
11.4 XGBoost Custom Loss Tuning 66
11.5 Linear Regression Rank Correction 68
11.6 SHAP Analysis Values 71

11.6.1 How each feature changes the rank from the baseline rank.....71
11.6.2 How the sum of the SHAP values across all features changes the rank of a given instance.....71
11.6.3 SHAP Value Code 72

Declaration of Authenticity

We hereby declare that we have completed the present thesis independently and without unauthorized assistance, that we have indicated all sources, literature, and other aids used, and that we have clearly marked all passages taken verbatim or in substance.

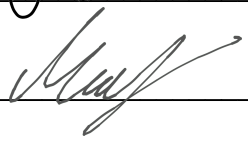
Declaration on the Use of AI Tools

For the preparation of this thesis, we also used the following generative AI tools (including ChatGPT and DeepL) for the following purposes: information search, support with text formulation, improvement of linguistic quality, and assistance with the structuring of content.

Barcelona, 18.07.25 

Barcelona, 18.07.25 

Barcelona, 18.07.25 

Barcelona, 18.07.25 

1. Executive Summary

This report presents the Search Engine Optimization Strategy for La Magnética, a digital agency in Barcelona. It covers the development and implementation of advanced Artificial Intelligence models, employing machine learning algorithms and neural network architectures, aimed at optimizing the Search Engine Optimization performance of client websites managed by La Magnética.

In light of the growing competition within digital markets, it has become essential for businesses to use data-driven methods to stand out online. To address this need, our group collaborated with La Magnética to support the development of predictive AI models tailored to their SEO strategy. Together, we built AI models that generate insights and spot patterns, helping the agency make clear, measurable improvements in their clients' search performance.

The core focus of this project is the development of custom AI models tailored for La Magnética's client, the official tourism portal *VisitNavarra.es*. These models were built with a specific predictive goal: estimating the search engine ranking position of individual pages for targeted keyword queries on Google Platform.

Through this project, we demonstrated how the integration of AI into the SEO workflow can significantly enhance strategic decision-making. For VisitNavarra.es, the implementation of these tools provides a scalable, objective mechanism for guiding SEO efforts and improving digital visibility. We kept improving the models using fine-tuning techniques and custom adjustments to make predictions more accurate and reliable. These refined models enabled more precise estimations of search engine ranking positions and allowed for more informed, data-driven decisions regarding keyword targeting, content updates, and technical SEO improvements. The optimization process not only increased model performance but also ensured that the outputs were actionable and aligned with real-world SEO priorities.

This project shows how advanced data science can make a real difference in digital marketing and provide clear business value.

2. Description of the company

La Magnética is an international digital strategy and marketing agency founded in 2009. The agency is based in Barcelona and serves clients in Spain, Germany, Netherlands, USA, Portugal and Greece. Its mission is to act as a strategic partner for companies and institutions by developing effective marketing strategies, implementing digital analytics and e-commerce solutions, and integrating technology to optimize performance. The agency envisions becoming a benchmark in Spain and Europe for intelligent, human-centred digital consulting, empowering clients to not only adopt digital tools, but also understand and use them effectively in their business processes. Its core values are being transparent, using data carefully, staying creative, adapting to change, working closely with clients, and focusing on results that can be measured. This helps make sure every action supports clear strategic goals.

La Magnética provides a broad set of services built on four main pillars: digital strategy and performance marketing, digital analytics and business intelligence, digital transformation and process optimization, and e-commerce and market intelligence. Under digital strategy, it offers Search Engine Optimization (SEO), Search Engine Marketing (SEM), inbound and outbound marketing, social media campaigns, and content strategy. Its analytics services include sector benchmarking, custom dashboards, Customer Relationship Management (CRM) integration, and advanced attribution modelling using tools such as Google Analytics, BigQuery and Tag Manager. For transformation, it supports clients in defining SMART goals, digitizing internal processes,

improving UX/UI design, and building data ecosystems. In e-commerce, the agency advises on marketplace positioning, sales channel development, and customer relationship digitization.

The agency serves a diverse portfolio of clients, including small and medium-sized enterprises, large corporations and public sector institutions. La Magnética is especially suited to organizations looking for strategic guidance to launch or scale digital marketing initiatives. It also helps align customer experience with business goals and adopt data ecosystems to transform operations...La Magnética stands out from larger or more traditional agencies thanks to its agile, specialized, and data-centric model, offering tailored solutions with a high degree of transparency and client collaboration. Ultimately, La Magnética combines technical expertise with strategic depth to deliver sustainable, intelligent, and human-centred digital solutions.

3. Aim of the Project

The primary aim of this project is to design, develop, and implement a predictive Artificial Intelligence framework tailored to improve the SEO performance of websites managed by the digital strategy agency La Magnética. The objective is to construct machine learning and deep learning models that estimate a web page's search engine ranking position using various content, structural, and behavioural SEO factors. The system is designed to produce accurate predictive outputs and also to support data-driven strategic decision-making for La Magnética and its clients. One example is the official tourism portal VisitNavarra.es. By integrating AI into the SEO workflow, this project seeks to automate the analysis of performance indicators, identify actionable optimization opportunities, and simulate the potential impact of SEO interventions. This contributes to the agency's broader mission of delivering intelligent, transparent, and results-oriented digital consulting.

4. Scope of the Project

Search Engine Optimization (SEO) refers to the process of improving the visibility of a website or individual web pages in the organic (non-paid) results of search engines like Google. The primary goal of SEO is to increase the quantity and quality of traffic to a website by improving its ranking for relevant search queries. SEO strategies typically involve a combination of technical, on-page, off-page, and user experience factors that affect how search engines index and rank content. A strong SEO strategy makes a website discoverable to users at key moments in their search journey, supporting business goals such as lead generation, brand awareness, and online conversions.

In SEO terminology, position refers to the numerical rank at which a webpage appears in the list of organic search results for a user's query. A lower number indicates a higher rank and greater visibility. Furthermore, results are typically displayed across multiple pages and appearing on the second page or later usually leads to a significant drop in visibility and click-through rates, which can have a strong impact on a business's online presence.

The data used to train and evaluate the models was provided directly by La Magnética and comes from two primary sources: Google Search Console (GSC) and Screaming Frog SEO Spider. GSC provides performance data such as impressions, clicks, position, and click-through rate (CTR) for specific keyword-page combinations, which provides insights into actual user behaviour and how search engines interact with the site. Screaming Frog offers a comprehensive crawl of a website's technical structure. While GSC provides performance data based on real user interactions with the site via Google Search, Screaming Frog offers a technical and structural analysis of the website's content and metadata. This combination makes it possible to correlate technical site attributes with actual ranking performance. This step is essential for building predictive SEO models. The scope includes the engineering of relevant features from technical SEO metrics,

page content attributes, and engagement indicators, followed by the development and training of multiple machine learning models, namely Random Forest, XGBoost, XGBoost with custom loss functions and deep neural networks. In addition to model building, the project includes the evaluation and tuning of model performance using metrics such as Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), Spearman correlation, Normalized Discounted Cumulative Gain (NDCG), and CTR-weighted accuracy. It also covers the implementation of SHAP, to ensure interpretability and transparency of predictions. Using supervised learning techniques alongside deep neural network modelling, we trained a system capable of predicting how various SEO interventions may influence search engine rankings. The resulting predictive framework allows La Magnética to test and evaluate different optimization strategies.

This project is limited to the organic search (non-paid) SEO domain and focuses specifically on modelling the ranking position as the output variable. It does not include the development of AI tools for paid advertising (SEM), social media analytics, or user experience optimization beyond search engine considerations. Additionally, while this model can be generalized to other domains, its training and evaluation are specifically scoped to the VisitNavarra.es site and its associated keyword-URL combinations.

In essence, the project covers the technical implementation including data preprocessing, custom feature engineering, model training, evaluation, and optimization.

5. State of the Art

5.1 Introduction

Search engine optimization (SEO) has gone through a lot of changes over the past years. At first, the focus was on placing keywords in strategic locations and getting large volumes of backlinks. These tactics were often enough already to improve rankings. However, search engines have become much more complicated nowadays. Google, for example, relies on advanced machine learning models to understand user intent and determine whether a page answers these search queries (Enge et al., 2022).

Because of that, SEO has changed into a much more challenging practice that combines different technical improvements, high-quality content, user experience considerations, and increasingly, data analysis and predictive modelling. That is why all of these areas need to be considered to achieve and sustain online visibility.

5.2 Modern SEO Methods

Google often updates its algorithms to improve the reliability and the relevance of search results. One of the most important updates in recent times was the introduction of BERT (Bidirectional Encoder Representations from Transformers), which allowed Google to understand the context of words in queries instead of just matching the exact keywords (Devlin et al., 2019). Another update was MUM (Multitask Unified Model), a natural language processing model, which improved that even more by allowing the search engine to process information at the same time from text, images, and also from multiple languages.

Credibility also has become an important ranking factor. Google ranks content using the so-called E-E-A-T principles (Experience, Expertise, Authoritativeness, and Trustworthiness). This is especially used for topics related to health, finance, or legal information (Enge et al., 2022). For

example, content about some medical advice is more likely to rank well if it is published by a recognized authority which has more credibility.

Despite these improvements and advancements from Google, technical SEO still remains important as well. Key elements include:

- Site speed: The faster the page loads, the better the user experience and the ranking performance.
- Mobile optimization: More and more searches nowadays happen on mobile devices which require different design choices.
- Structured data: Schema markup helps search engines to interpret content and display rich snippets.
- Clean URL structures: Consistent and descriptive URLs help to faciliate crawling and indexing.
- Duplicate content prevention: Avoiding repeated content helps to reduce the confusion for search engines and therefore prevents reducing ranking.

In the context of SEO measurement and reporting, different metrics are used to determine visibility and performance. These definitions are based on well-known practices described in The Art of SEO (Enge et al., 2022) and the reporting functionality in Google Search Console.

- Clicks: The total number of times users have clicked a search result and visited the website. Clicks are the traffic generated from organic searches.
- Impressions: The number of times a URL from the website has appeared in search results, even if it was not clicked. Impressions show the frequency with which content is shown to users.

- Positions: This metric shows the average ranking of a page or keyword in Google Search results. This means the first search result is position 1 on the first page and so on. This measure is considered the most important in analysis.
- Click-Through Rate (CTR): The percentage of impressions that result in a click. CTR is calculated by dividing the number of clicks by the number of impressions.

These indicators are important to understanding SEO performance and are the basis for analysing improvements that come from optimization efforts (Enge et al., 2022).

Keyword strategies have changed more to long-tail keywords, which are more specific and typically have lower competition. These queries often show stronger intent to purchase or engage. For example, a phrase such as *"best CRM software for small businesses in Spain"* provides clearer intent than *"CRM software."*

Data privacy regulations have also changed some SEO practices. For example, the introduction of GDPR (*Regulation - 2016/679 - EN - Gdpr - EUR-Lex*, n.d.) required companies to reconsider how data is collected and stored. This forced companies to get first-party data through user consent.

5.3 AI and Machine Learning in SEO

Machine learning can bring new improvements to SEO. Predictive models can estimate future outcomes and therefore help businesses make the right decisions.

Some machine learning models that can be used for that purpose are:

- Random Forest: An ensemble method that builds multiple decision trees and sums their outputs (Breiman, 2001). Random Forest models are often used for their easy interpretation, as they can highlight the most important factors that lead to the predictions.

- XGBoost: A gradient boosting algorithm that builds trees sequentially, and therefore improves prediction accuracy with each iteration (Chen & Guestrin, 2016). XGBoost has become a very popular choice in practical applications because of its efficiency and strong performance on structured data.
- Deep Learning: Neural networks can model complex relationships in text as well as numeric data. In SEO, deep learning models can be used for content classification, semantic similarity assessment, and also traffic forecasting. Embeddings can also provide a way to measure how closely a page aligns with a user query (Devlin et al., 2019). However, for these models to work, they require larger datasets and also more computational power.

SHAP (SHapley Additive exPlanations) provides a consistent way to interpret machine learning model outputs by assigning each feature a contribution value. It calculates how much each input increases or decreases a prediction compared to a baseline (Awan, 2023). SHAP is especially useful for explaining tree-based models such as Random Forest and XGBoost. In SEO, it helps clarify why the model predicts certain rankings or click-through rates, supporting transparent reporting and more informed decisions (*Search Console Help*, n.d.).

Combining these machine learning models with already established SEO metrics can improve decision-making for businesses. To assess the accuracy and relevance of model predictions, different evaluation metrics are used. Some of these common evaluation metrics for machine learning models are the following (Hyndman and Athanasopoulos, 2018):

- Root Mean Squared Error (RMSE): RMSE measures the average of prediction errors, where larger errors are penalized more heavily. Lower RMSE values indicate that predictions are closer to observed or desired results.

$$RMSE = \sqrt{\sum_{i=1}^{n} \frac{(\hat{y}_i - y_i)^2}{n}}$$

Formula 1. Root Mean Squared Error (RMSE)

- Mean Absolute Error (MAE): MAE calculates the average of absolute differences between predicted and actual values. This metric provides a simple interpretation of overall accuracy without looking at outlier errors as strongly as RMSE.
- Spearman Correlation: Spearman correlation evaluates the rank-order relationship between predicted and actual values. In SEO, this metric helps to understand whether a model correctly sees the relative order of pages or keywords in ranking predictions. Scores range from -1 (perfect negative correlation) to 1 (perfect positive correlation), with 0 indicating no correlation. In SEO, the goal is to maximize this value so predicted rankings closely match the observed rankings.

$$\rho = 1 - \frac{6\Sigma\, \mathrm{d}_i^2}{n(n^2 - 1)}$$

Formula 2. Spearman's Rank Formula

- Normalized Discounted Cumulative Gain (NDCG): NDCG measures the quality of ranking predictions by placing more weight on correct ordering at higher positions. This is important because higher-ranked results typically receive most clicks. Scores are

normalized between 0 and 1, with higher values indicating better performance (Järvelin and Kekäläinen, 2002).

The combined use of these metrics gives a balanced overview of model performance across numeric prediction accuracy, ranking quality, and expected business impact.

5.4 Tools and Technologies

Current SEO workflows use established tools, but a custom analysis is always needed as well. Some of these already established tools are:

- Google Search Console: The primary platform for monitoring keyword performance, indexing status, as well as technical errors (*Search Console Help*, n.d.).
- Screaming Frog: A website crawler that can identify issues such as missing metadata, broken links, and duplicate content (‘Screaming Frog SEO Spider Website Crawler’, n.d.).
- Python libraries: scikit-learn, XGBoost, and TensorFlow are used for model development (Chen & Guestrin, 2016). scikit-learn is suitable for simpler models, while TensorFlow can be used to build and train deep neural networks.

Using multiple tools help to create a more complete understanding of SEO performance. For example, combining crawl data with keyword impressions help with the analysis of how technical improvements are connected to ranking trends.

5.5 Competitors and Market

AI-powered SEO is becoming more common, with different SaaS (Software as a Service) platforms and agencies using machine learning in their services. Some examples of specialized tools and platforms include:

- SurferSEO: Provides NLP-based recommendations to optimize content structure and relevance.

- Clearscope: Offers content scoring and keyword suggestions from competitive analysis.
- MarketMuse: Supports content planning and evaluation by highlighting gaps in the content.

In Spain, agencies such as BigSEO (Barcelona) and Internet República (Madrid) already combine automation with SEO consulting (Enge et al., 2022). The growth of these solutions indicates that machine learning is becoming a standard component of competitive SEO strategies.

One potential advantage for La Magnética is the ability to develop tailored solutions that integrate standard tools with custom models, offering clients more flexibility and detailed insights compared to purely off-the-shelf software.

5.6 Conclusion

Current SEO strategies combine well-established practices with new machine learning techniques. Foundational elements such as strong technical architecture, high-quality content, and a clear site structure remain critical (Enge et al., 2022). However, predictive models including Random Forest (Breiman, 2001), XGBoost (Chen & Guestrin, 2016), and deep learning networks (Devlin et al., 2019), enable more accurate forecasting and evidence-based recommendations.

Using these methods supports a transition from descriptive reporting toward proactive, data-driven strategies. This approach is expected to enhance La Magnética's service offering and improve clients' long-term search performance.

6. Development or Body of the Report

The development of the AI model that we built followed a structured pipeline involving data preprocessing, feature engineering, model training, and evaluation.

6.1 Data Preprocessing

The data preprocessing phase began with importing two datasets: `url_kw_table.xlsx` and `raw_urls.xlsx`. The first data `url_kw_table.xlsx` corresponds to data from Google Search Console (GSC), which includes keyword performance metrics such as impressions, clicks, CTR, and position which are core data points provided by GSC. In this dataset, the main dimension is the combination of URL and keyword, meaning that each row represents the SEO results for a specific keyword driving traffic to a specific URL. Meanwhile, `raw_urls.xlsx` originates from Screaming Frog SEO Spider. Screaming Frog is used for crawling website URLs and collecting on-page technical data such as metadata (title, meta description), H1/H2 tags, status codes, and content length.

These two datasets were merged on URL-related fields to form a unified dataset. Missing values were handled by eliminating incomplete rows and dropping empty columns, ensuring the integrity of the dataset. Features that contained text or categories were first checked to see how many unique values they had. Then, these features were converted into a numerical format using a method called one-hot encoding, which creates a new column for each category so the model can understand and use them during training. Furthermore, numerical columns that were stored as strings with commas as decimal points were standardized and converted to float types. This preprocessing was critical to prepare the data for robust machine learning algorithms, ensuring numerical consistency and eliminating noise that could skew model learning.

6.2 Transforming and Encoding

As keywords are so important to a website performance in the rankings, we wanted a way to be able to capture this in our model. Therefore, we started by adding features that measure the presence of the keyword in a site feature. This was done on a simple scoring basis. The fraction of the keyword that appeared in a site feature was given as a score for that feature.

Additionally, the semantic similarity of different site features and keywords was an additional set of features added to the data. A pre-trained AI language model called SentenceTransformer was used, which enabled the ability to capture the meaning of the keywords and site features by creating vector embeddings of each keyword and site feature. The SentenceTransformer creates vector embeddings that are 384 values in length. These numbers capture the meaning of the text that has been passed into it by encoding each of the tokens and then aggregating their combined vectors into one overall embedding for the entire text (Gupta, 2025).

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}},$$

Formula 3. Spearman's Rank Formula

Using the cosine similarity formula shown in formula 3, the semantic similarity could then be measured using cosine similarity between the embeddings. The cosine similarity value ranges from -1 to 1:

- **1**: Indicates that the two vectors are the same.
- **0**: Indicates that the two vectors are orthogonal (they have no similarity).
- **-1**: Indicates that the two vectors are exactly opposite.

Other encoders could have been used to create the embeddings, but then would have encoded the keywords and site features on a token level instead of a sentence level and so would have exploded the size of the dataset. (See Technical Annex 11.1 for more detail).

6.3 Model Training

In order to predict search engine ranking positions, both baseline and advanced machine learning models or more especially, Random Forest and XGBoost regressors, were implemented during the project's model training phase. To ensure that the models performed well across various keyword groups, the dataset was first divided using GroupKFold cross-validation to stop keyword leakage between training and test sets. With sample weights applied based on impression volume and rank sensitivity, feature matrices (X) and target values (y) representing average Google Search positions were generated.The Random Forest model was trained using standard mean squared error as the objective function, leveraging bootstrap sampling and feature subsampling to prevent overfitting. In parallel, a custom loss function was developed for XGBoost to better reflect business-specific goals. This loss function incorporated a CTR-weighted mean squared error with additional penalties for mis-ranking high-priority (top-ranked) keywords. A contrastive weighting mechanism further emphasized ranking accuracy at the top of search engine results. This weighting reflects the typical CTR curve observed in search results, where click-through rates drop steeply from the first positions and gradually level off on lower ranks. The XGBoost model was trained using this custom objective and evaluation metric, with early stopping based on performance on a held-out validation set. Predictions from both models were clipped to maintain plausible ranking values, and final model outputs were stored for evaluation. This training

process ensured the development of robust models that aligned both with technical performance and strategic SEO impact.

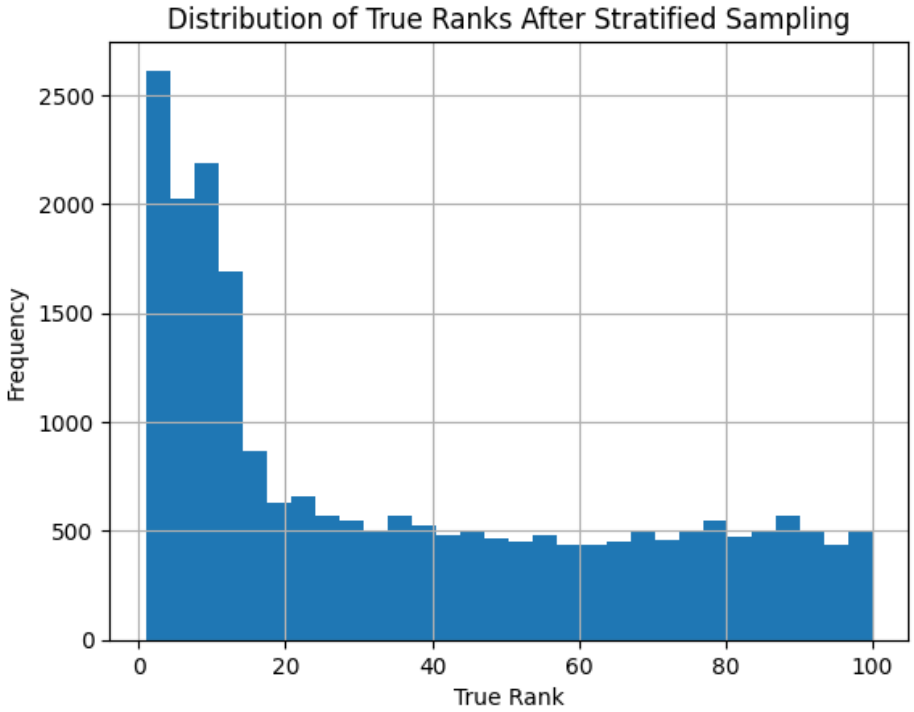


Figure 1. Distribution of True Ranks After Stratified Sampling

Figure 1 is a histogram that illustrates the frequency distribution of actual search engine ranking positions in the dataset following the application of stratified sampling. The x-axis represents the true rank positions (ranging from 1 to 100), while the y-axis shows the frequency of occurrences for each ranking interval. As shown, there is a high concentration of data points in the top ranks (positions 1–10), which gradually decreases until around position 30. After that point, the distribution becomes more uniform across the remaining ranks. This indicates that stratified sampling successfully balanced the representation of both high-ranking and lower-ranking URLs in the training data, preventing overrepresentation of top-ranking keywords while ensuring that all ranking strata are adequately learned by the model. This distribution is critical for training models that generalize across a wide range of ranking outcomes.

6.3.1 Evaluation Function

In the evaluation of predictive models, particularly those applied to ranking and regression tasks such as click-through rate (CTR) prediction, a combination of performance metrics is essential to comprehensively understand model behavior. Four key metrics were employed to measure different aspects of model accuracy, robustness, and real-world applicability.

Key Metrics:

1. Root Mean Squared Error (RMSE)
2. Mean Absolute Error (MAE)
3. Spearman's Rank Correlation Coefficient
4. Normalized Discounted Cumulative Gain (NDCG)

The combined use of these metrics ensures a well-rounded assessment: RMSE and MAE for error distribution and numeric prediction accuracy, Spearman for rank preservation, NDCG for relevance-based ordering or ranking quality of the outputs. However, in addition to this two additional evaluation metrics were defined to allow for a more task specific evaluation of the model's performance:

5. CTR-Weighted Evaluation
6. Root Mean Squared Error (RSME) by Rank Bin

The CTR-weighted Accuracy was a custom ranking score inspired by the NDCG metric. However, this metric was tailored to follow the CTR weighting mentioned above, that had been defined from previous work. Like the NDCG this gave more importance to correctly ranking keywords that are likely to bring the most clicks. It does this by using a curve that weighs top positions more strongly. As a result, if the model predicts the most valuable keywords correctly at the top, it receives a better score. However, unlike the NDCG, this score can be estimated from historical click data of

a specific business, so it better reflects real-world engagement patterns. This approach ensures that the evaluation focuses on what matters most for driving traffic and conversions.

The RSME of each rank bin was also used as an evaluation metric, as having been found in the CTR weighting, the difference between a second and fourth ranked URL-Keyword pair is much higher than that of the twentieth and fortieth. Therefore, it was necessary to more closely analyse the error at the top ranks and to be able to specifically minimize these metrics over reducing the RSME of the entire model. (See Technical Annex 11.2 for more detail).

6.3.2 Machine Learning Application

6.3.2.1 First Model: XGBoost and Random Forest

Traditional machine learning models were utilized as baselines for comparison. These included Random Forest and XGBoost. These models offered fast training and interpretability, making them suitable for preliminary analysis. By comparing their performance against more advanced models, their limitations in handling complex, nonlinear interactions were identified. This analysis justified the pursuit of more sophisticated models such as XGBoost and neural networks.

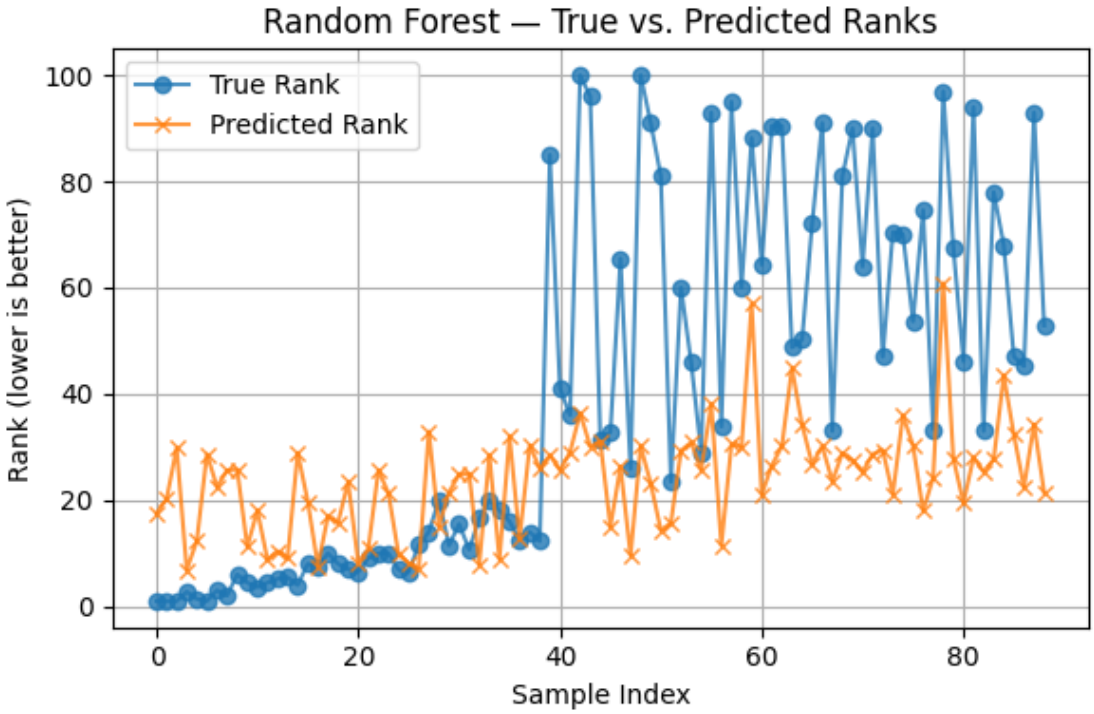


Figure 2. First Model Prediction for Random Forest

Figure 2 displays the Random Forest predictions compared with the real search engine ranking positions. The predictions show big overestimation, especially for pages with an actual rank of 1, and high underperformance with high ranks. This consistent overprediction which is evident from errors close to low and really high true positions. These results highlight a critical challenge in the model's calibration, emphasizing the need for more robust error correction strategies or better handling of high rank values during training to improve accuracy and reliability in high-impact SEO decisions.

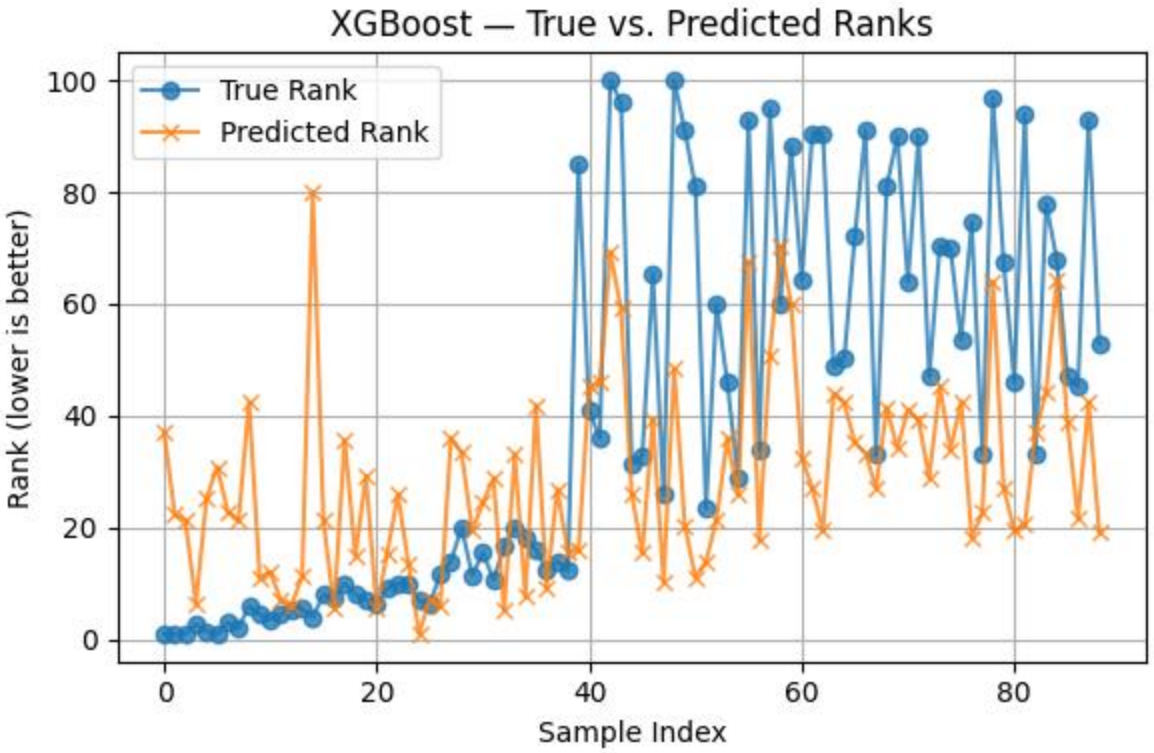


Figure 3. First Model Prediction for XGBoost

Figure 3 shows the predictions generated by the XGBoost model. The model consistently overpredicts the rank, especially for pages with true ranks close to 1, such as positions 1-10 where the predicted positions are significantly higher. This trend is a major challenge in the model's performance. It struggles to correctly estimate highly ranked or top-performing pages, often assigning them much lower predicted relevance than warranted. Although the model performs slightly better in higher positions, the overall pattern indicates a bias toward underestimating the visibility of top-ranking URLs. This finding underscores the importance of refining the model's calibration and loss function, particularly in scenarios where high-ranking accuracy is critical for SEO decision-making.

6.3.2.1.1 First Model Evaluation

	Model	RMSE	MAE	Spearman	NDCG@10	CTR-Weighted@10	RMSE_1–3	RMSE_4–6	RMSE_7–10	RMSE_11–20	RMSE_21+
0	Random Forest	912.4689	22.8929	0.4529	0.0144	2.4150	413.8421	189.4602	82.2586	133.2479	1807.9256
1	XGBoost	838.9061	22.1985	0.4383	0.0342	2.5917	551.9231	1036.6646	156.7313	187.3951	1381.0023

Table 1. First Model Evaluation

Table 1 presents a comparative evaluation of two machine learning models Random Forest and XGBoost based on a range of ranking performance metrics. XGBoost outperforms Random Forest in overall RMSE (Root Mean Square Error), with a lower value of 838.91 compared to 912.47, indicating better predictive accuracy. Both models show similar Mean Absolute Error (MAE), with XGBoost slightly better at 22.20 versus 22.39. In terms of Spearman correlation, which measures the monotonic relationship between predicted and actual rankings, Random Forest slightly leads with 0.4529 over 0.4383. However, XGBoost significantly improves the NDCG@10 score (Normalized Discounted Cumulative Gain), which evaluates the quality of top-ranked results, achieving 0.0342 compared to Random Forest's 0.0144. Similarly, XGBoost performs better in CTR-weighted accuracy at the top 10 positions (2.5917 vs. 2.4150), emphasizing its strength in prioritizing high-impact predictions. Although XGBoost has better overall RMSE score, we can see that Random Forest performs better in the top positions (pages 1–10), while XGBoost performs better in lower-ranking positions (11+).

6.3.2.2 Second Model: XGBoost Custom Loss

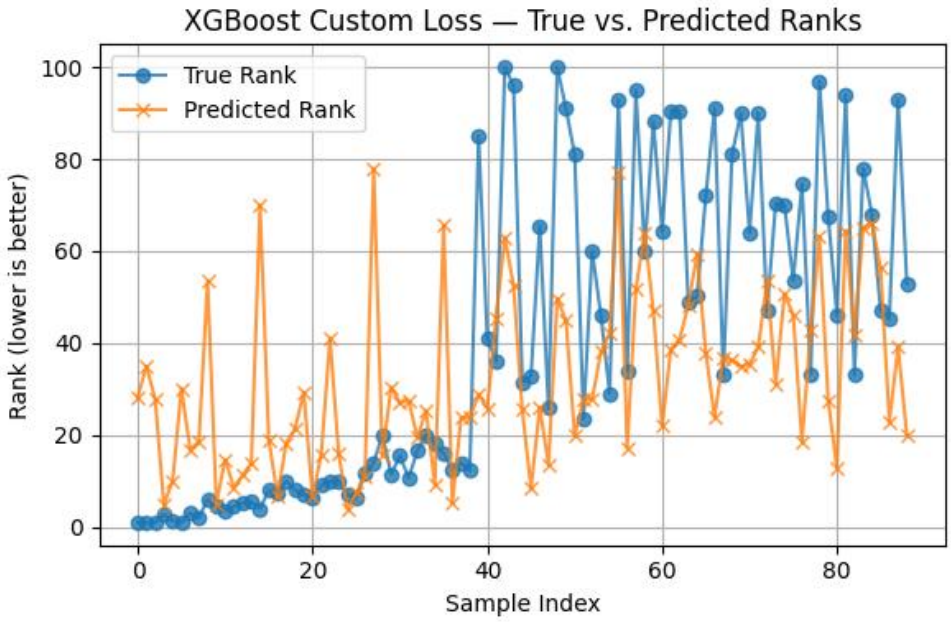


Figure 4. Line plot for XGBoost with Custom Loss

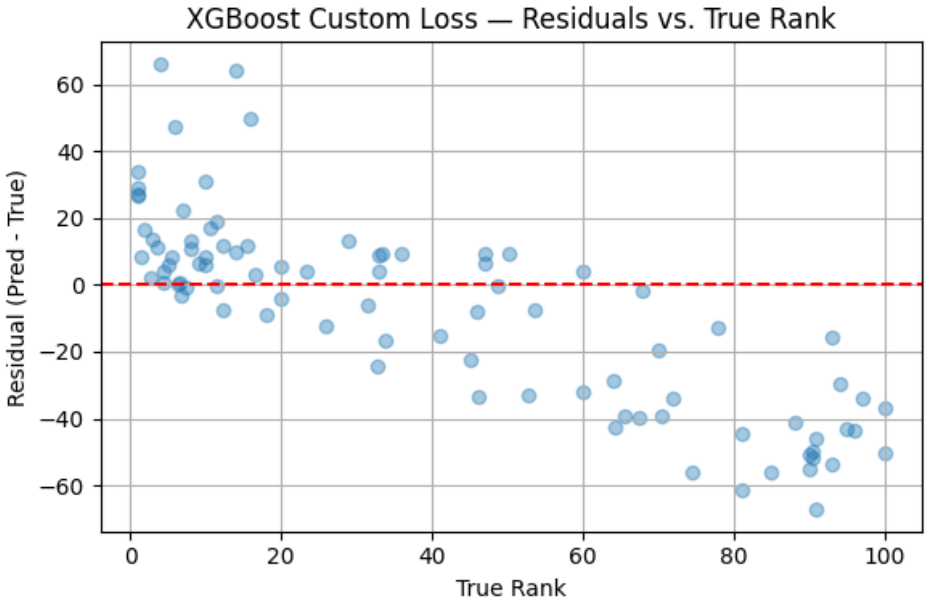


Figure 5. Scatter Plot for XGBoost with Custom Loss

Figures 4 and 5 illustrate the performance of the XGBoost model trained with a custom loss function specifically designed to emphasize the importance of high-ranking positions. it can be seen that the custom loss leads to more stable predictions across a range of samples, with reduced fluctuations compared to untuned versions. Nonetheless, there remains a tendency to underestimate the highest-ranked pages.

The residual scatter plot shows that residuals systematically decrease as true rank increases. This indicates that for lower-ranked pages (higher numerical ranks), the model tends to underestimate predicted positions, whereas for top-performing pages, it overestimates.

While the overall RMSE is slightly higher when compared to Random Forest (as shown in the evaluation table), the custom loss function demonstrates improved performance in other metrics, such as CTR-weighted accuracy and NDCG@10. This suggests that despite certain limitations in absolute rank prediction, the custom loss better aligns the model with practical business objectives focused on maximizing user engagement and prioritizing high-impact positions.

6.3.2.2.1 CTR Curve

A Click-Through Rate (CTR) curve was developed to visualize the relationship between predicted relevance scores and actual user engagement. This curve helped validate the effectiveness of the model in ranking pages by expected user interaction. It served as a practical tool to interpret the model’s impact beyond raw numerical scores.

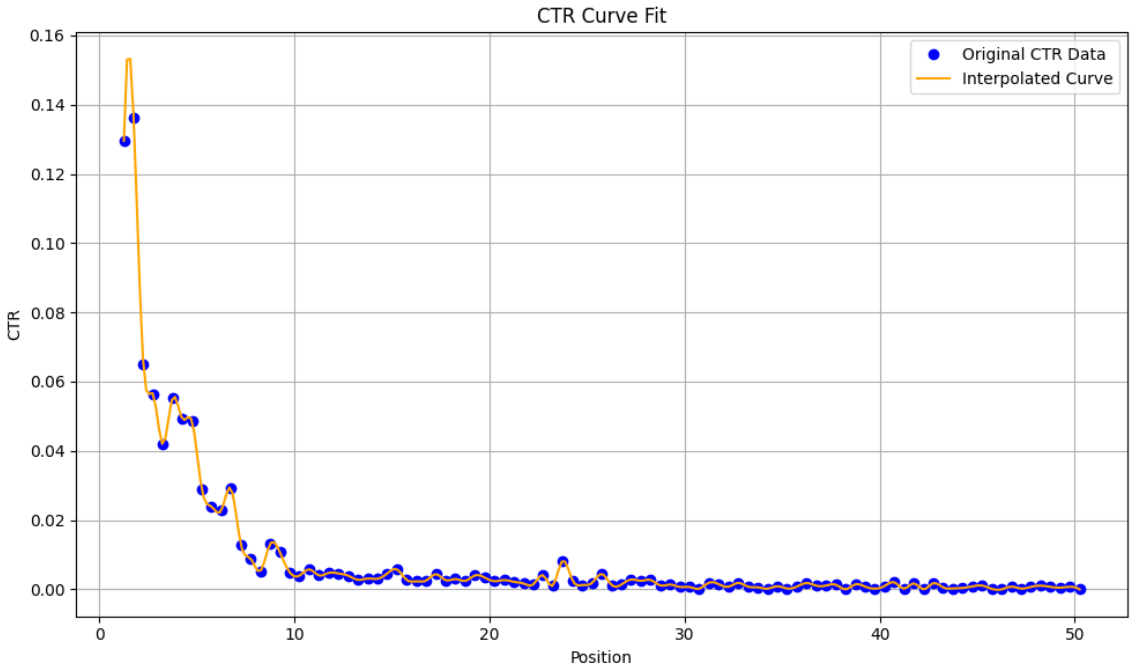


Figure 6. CTR Curve

In figure 6 the relationship between a webpage's position in search engine results and its corresponding click-through rate (CTR) is illustrated. The blue dots represent the original CTR data points, showing the observed CTR values at each position from 1 to 50. The yellow line is a smooth, interpolated curve fitted to these points to model the overall trend. As shown in the plot, pages that rank higher (closer to position 1) receive significantly more clicks, with CTR sharply declining as the position number increases. This steep drop highlights the importance of securing a top-ranking position on search engines. The fitted curve provides a clearer understanding of this trend and can be used for weighting model predictions or evaluating ranking improvements.

6.3.2.2.2 CTR VS Fitted Power-Law weighting Function

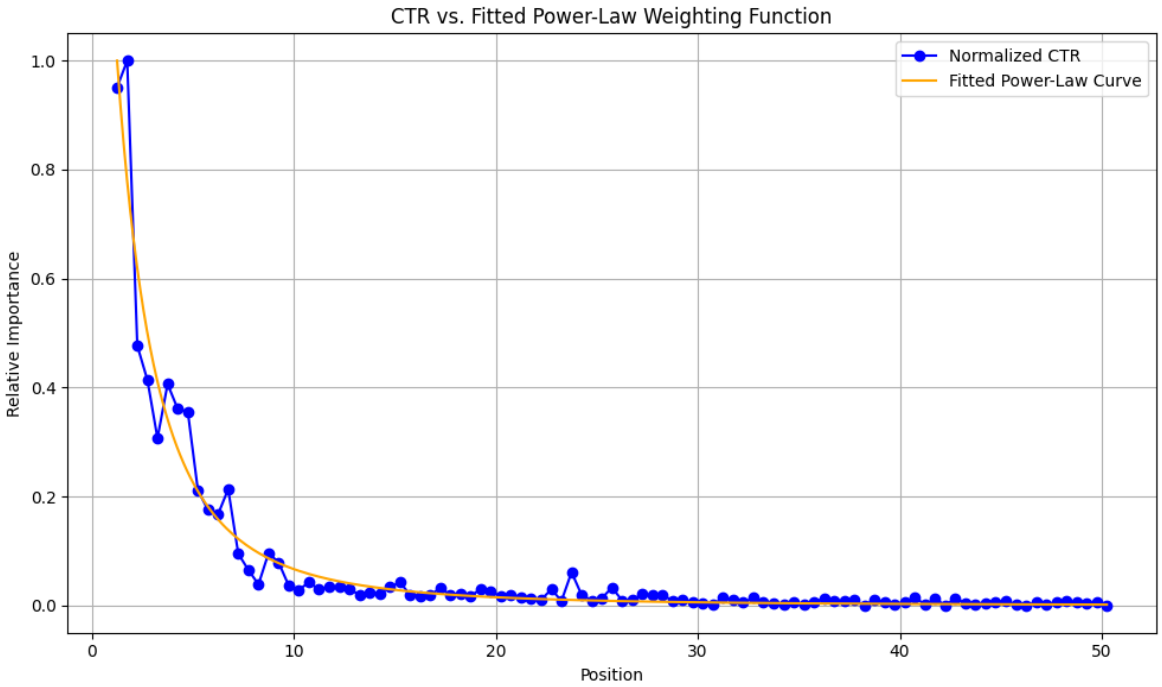


Figure 7. CTR vs Fitted Power-Law Weighing Function

Figure 7 visualizes the relationship between a webpage's position in search engine results and its relative importance, using normalized click-through rate (CTR) as a proxy. The blue line and dots represent the normalized CTR values across different positions, showing how user engagement diminishes the lower a page appears in the rankings. The orange curve represents a fitted power-law function, which models this decline mathematically. The close alignment of the curve with the actual data suggests that a power-law distribution effectively captures the drop-off in importance. This function was used as a weighting mechanism in the model evaluation to reflect how position affects user behaviour—emphasizing the importance of higher-ranking positions when predicting and optimizing search rankings.

6.3.2.2.3 Creating a Custom Loss Function

The process of creating a custom loss function required manual definition of gradients and hessians, ensuring compatibility with the XGBoost training API. This advanced customization allowed precise control over the model's learning behaviour. It emphasized penalizing incorrect rank positions more heavily, which was beneficial for ranking accuracy in CTR prediction. (See Technical Annex 11.3 for more detail).

6.3.2.2.4 Distribution of Log-Transformed Search Volume Weights

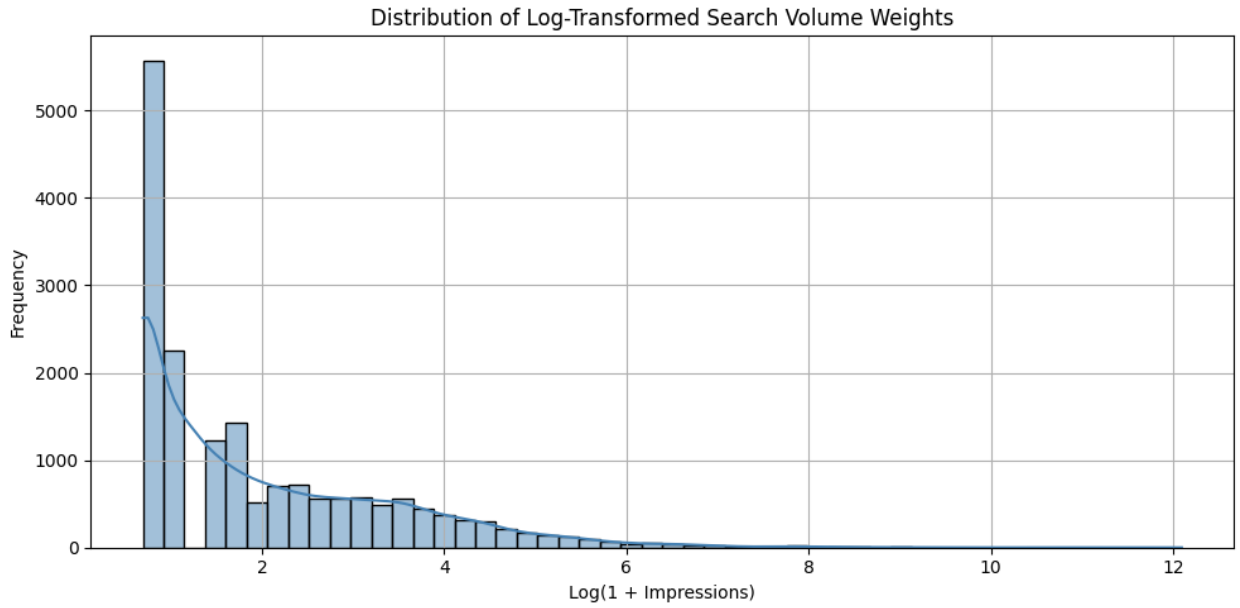


Figure 8. Distribution of Log-Transformed Search Volume Weights

Figure 8 shows the distribution of search volume weights after applying a logarithmic transformation to impression data (log(1 + impressions)). The x-axis represents the transformed values of search impressions, a common practice to reduce skewness caused by a few extremely high values. The y-axis indicates the frequency of keywords or queries at each level of transformed impression volume. As seen in the histogram, the majority of keywords have relatively low impression counts, resulting in a right-skewed distribution. The log transformation

helps compress the scale, allowing the model to account for the importance of high-volume queries without letting them disproportionately dominate the training process. This transformation was used to weight training samples, ensuring that both high- and low-volume keywords contribute meaningfully to the learning process.

6.3.2.2.5 Second Model Evaluation

	Model	RMSE	MAE	Spearman	NDCG@10	CTR-Weighted@10	RMSE_1–3	RMSE_4–6	RMSE_7–10	RMSE_11–20	RMSE_21+
0	Random Forest	725.8789	22.4574	0.4634	0.1938	2.3003	1259.9988	587.0146	364.6142	532.7218	957.1372
1	XGBoost Custom Loss	819.2133	22.0847	0.4469	0.0226	1.8262	500.7832	979.4857	174.4650	596.6075	1177.0495

Table 2. Second Model Evaluation

Table 2 compares the performance of two models: Random Forest and XGBoost with a Custom Loss Function based on a variety of ranking and regression metrics. The Random Forest model achieved better overall error metrics in terms of RMSE (725.88) and NDCG@10 (0.1938), suggesting it more accurately predicted precise ranking positions and preserved the relevance order for top-ranked URLs. However, the XGBoost Custom Loss model, while slightly worse on RMSE and NDCG@10, performed notably better on CTR-Weighted@10 (1.8262 vs. 2.3003 for Random Forest) and RMSE for the critical top-3 positions (500.78 vs. 1259.99).

This indicates that XGBoost Custom Loss, despite a higher overall error, was more effective at prioritizing performance on the top-ranking positions that matter most for visibility and clicks. This trade-off reflects the model's alignment with real-world SEO goals, emphasizing business impact rather than only raw accuracy.

6.3.2.3 Third Model: Fine-Tuning XGBoost Custom Loss Function

The custom loss function was incorporated into a tuned XGBoost model. Hyperparameter optimization was conducted to extract maximum performance from the model. Evaluation results demonstrated improved ranking metrics, confirming the effectiveness of the customized objective function and validating its practical utility.

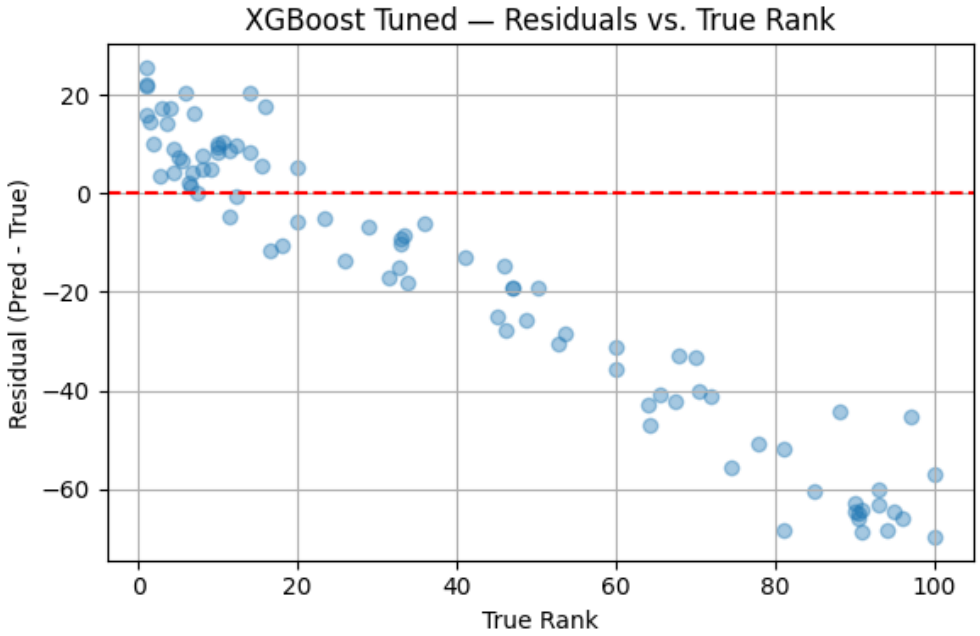


Figure 9. Scatter Plot of Third Model Predictions for XGBoost Tuned

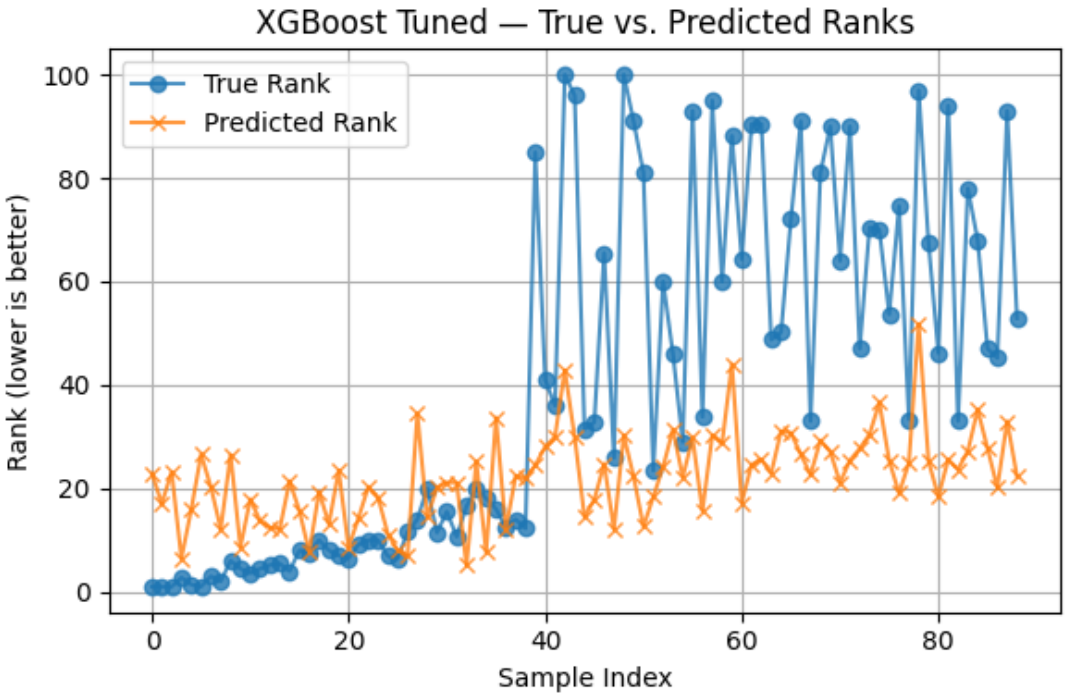


Figure 10. Line Plot of Third Model Predictions for XGBoost Tuned

These plots (figures 9 and 10) show the predictions from the XGBoost model after hyperparameter tuning and compared with real ranking values. In this project, the XGBoost model was tuned by adjusting key hyperparameters, including:

- **max_depth** - to control the maximum depth of each tree, affecting model complexity,
- **learning_rate** - to control how quickly the model adapts to errors,
- **Subsample -** to control the fraction of samples
- **Colsample_bytree** -to control features used in each tree to prevent overfitting,
- **Reg_alpha (L1) -** to make the model generalize better and reduce overfitting
- **Reg_lambda (L2) -** to make the model generalize better and reduce overfitting.

Compared to earlier versions, the tuned model demonstrates slightly improved accuracy on several observations, for example, a significantly lower error on lower rankings (1 - 20), while having worse ranking on higher true ranks (21+). As we can see for top-ranked pages (those with the lowest numerical rank, highest visibility), the model consistently underestimates and assigns them worse ranks than they truly have. This shows that while the model is effective at general ranking estimation, it struggles to capture important distinctions to accurately predict the top positions.

While the tuning process led to some refinement and increased prediction stability, particularly for mid-range rankings, it remains evident that there is room for further model enhancement, especially in accurately predicting low-performing SEO content where rank sensitivity is critical. (See Technical Annex 11.4 for more detail).

6.3.2.3.1 Third Model Evaluation

	Model	RMSE	MAE	Spearman	NDCG@10	CTR-Weighted@10	RMSE_1–3	RMSE_4–6	RMSE_7–10	RMSE_11–20	RMSE_21+
0	Random Forest	725.8789	22.4574	0.4634	0.1938	2.3003	1259.9988	587.0146	364.6142	532.7218	957.1372
1	XGBoost Custom Loss	819.2133	22.0847	0.4469	0.0226	1.8262	500.7832	979.4857	174.4650	596.6075	1177.0495
2	XGBoost Tuned	972.4780	23.3823	0.4595	0.0791	2.4689	313.3950	161.0137	60.8097	110.0880	1943.2537

Table 3. Third Model Evaluation

Table 3 compares the performance of three models: Random Forest, XGBoost with a custom loss function, and a tuned XGBoost model for predicting search engine rankings. While Random Forest delivers strong overall metrics, including the lowest total RMSE (725.88), highest NDCG@10 (0.1938), and best Spearman correlation (0.4634), it struggles in the most critical ranking positions. Its RMSE in the top 1–3 ranks is the highest of the three models at 1259.99, and it remains relatively high across ranks 1–10, which suggests it is less reliable when accuracy at the very top of the search results matters most.

XGBoost with a custom loss function performs significantly better in that area, achieving the lowest RMSE for the top 3 ranks (500.78) and a strong CTR-weighted score, making it more effective for prioritizing high-visibility results. The tuned version of XGBoost is even more accurate in the mid-range ranks (4–10), with RMSE as low as 161.01 and 60.81 in those bands. However, it suffers from poor generalization, with the highest overall RMSE (972.48) and a steep performance drop-off in the lower ranks (RMSE_21+ = 1943.25).

In summary, Random Forest remains the most well-rounded model overall, but if the goal is to optimize accuracy at the very top of the rankings, XGBoost with a custom loss may be the better choice. The tuned XGBoost model excels in the middle but at the cost of broader consistency. The ideal model really depends on which part of the ranking spectrum matters most.

6.3.2.4 Fourth Model: XGBoost Tuned + Rank-Based Correction

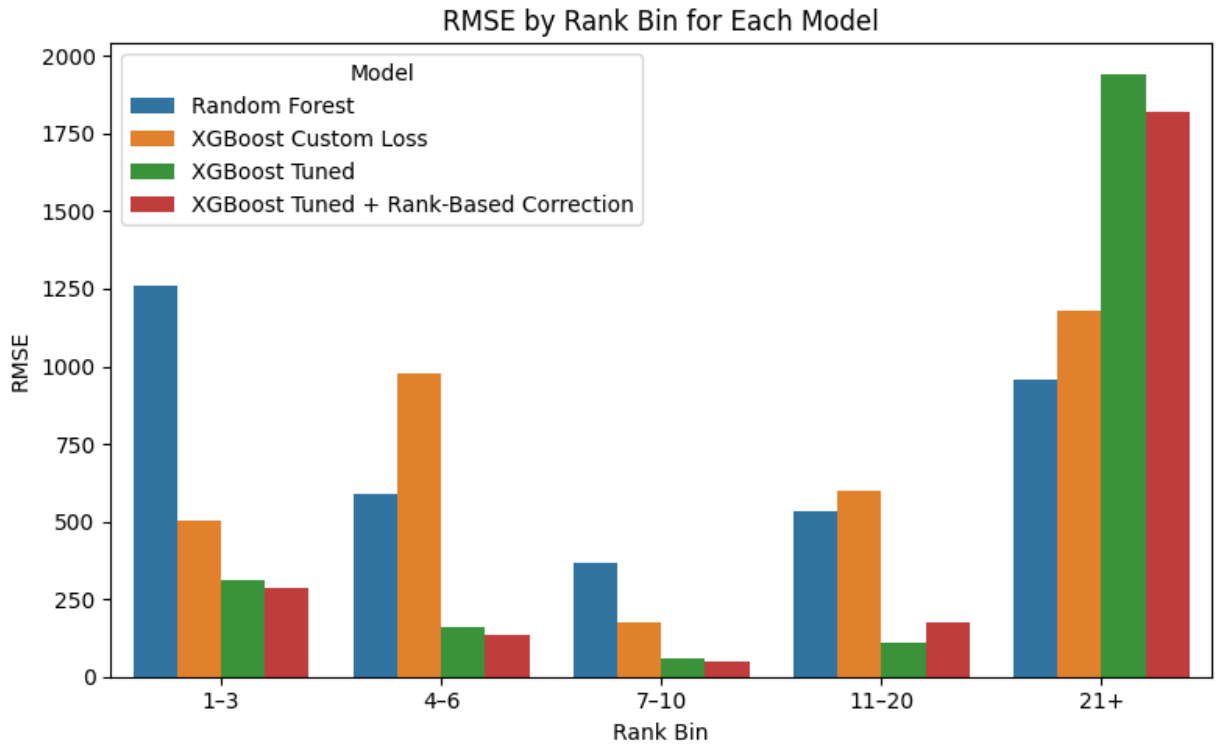


Figure 11. RMSE by Rank Bin for Each Model

Figure 11 shows how four models perform in predicting search result rankings, using RMSE across five rank bins: 1–3, 4–6, 7–10, 11–20, and 21+. Lower RMSE means better accuracy.

XGBoost Tuned with rank-based correction clearly leads in the top ranks (1–6), with the lowest errors across those bins. The regular tuned version also performs well here. In contrast, Random Forest has much higher RMSE in the top 1–3 range, making it less suitable when top results are prioritized. XGBoost with custom loss starts strong in ranks 1–3 but performs poorly in ranks 4–6, suggesting it is less consistent overall. In the middle ranks (7–20), the two tuned models continue to show strong, stable performance. Random Forest and the custom-loss model again fall behind, with noticeably higher error. In the 21+ range, things shift: Random Forest outperforms

all others with the lowest RMSE, followed by the custom-loss model. The tuned models, particularly the uncorrected one, see a sharp increase in error here.

Overall, XGBoost Tuned with rank-based correction delivers the best accuracy especially at ranks 1–10 which is the highest priority, while Random Forest still holds an advantage in the long tail. The custom-loss model shows promise at the top but lacks consistency across the board.

6.3.2.4.1 Applying a Linear Regression Correction

To refine the final predictions, a linear regression model was trained on the residuals of the XGBoost outputs. This secondary model captured any systematic errors not addressed by the main predictor. The correction was then applied to new predictions to enhance overall accuracy and calibration.

6.3.2.4.2 Validate Residual Correction to Avoid Overfitting

Residual corrections were validated on an unseen test set to ensure generalization. This step was vital to avoid overfitting, which could result from applying the correction too specifically to training data. Successful validation confirmed that the residual correction improved performance without introducing bias or overfitting.

6.3.2.4.3 Overall Model Predictor Class

A comprehensive predictor class was developed to encapsulate the entire modeling workflow: data preprocessing, model training, prediction, and post-prediction correction. This modular class structure enhanced reusability, maintainability, and clarity. It allowed seamless execution of the full pipeline from raw input to final predictions.

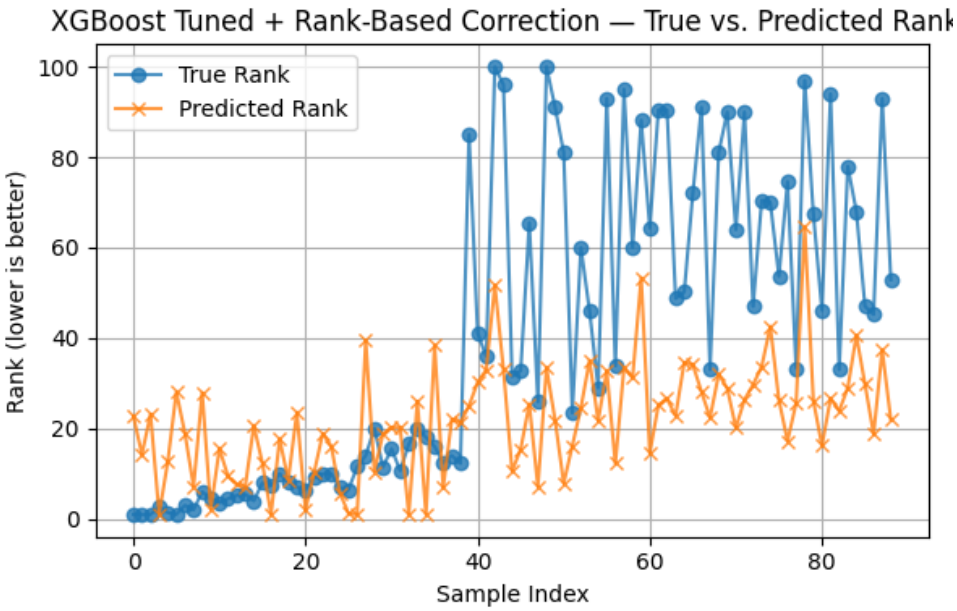


Figure 12. Line Plot of XGBoost Tuned + Rank-Based Correction

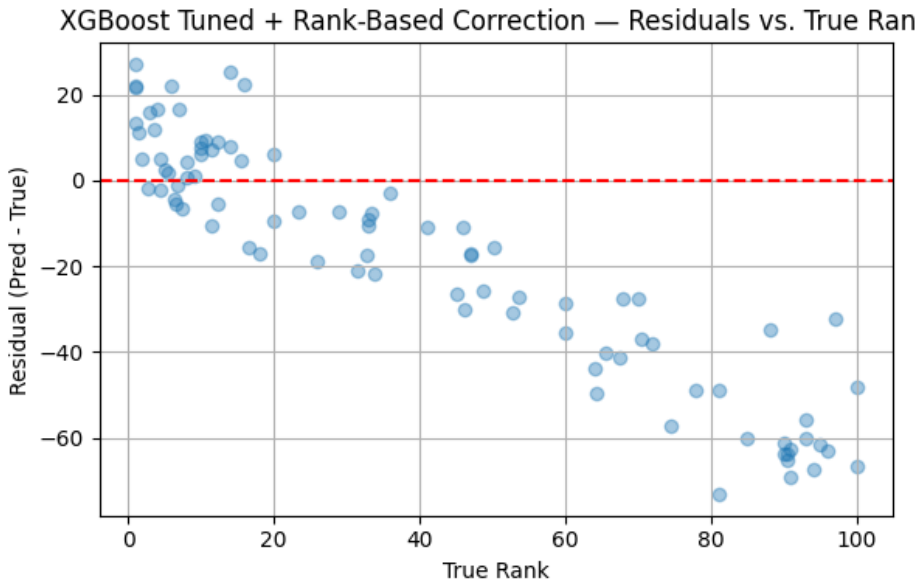


Figure 13. Scatter plot of XGBoost Tuned + Rank-Based Correction

The XGBoost Tuned model with Rank-Based Correction (figures 12 and 13) represented the most advanced approach developed in this project. After extensive testing, XGBoost was selected for

its strong performance in handling structured feature data, its ability to capture non-linear relationships, and its flexibility when integrating custom loss functions.

In this project, the model was further improved by adding a rank-based correction layer, designed to address systematic biases observed in the raw predictions. For example, the original tuned model tended to overestimate the rank for the top-performing pages and underestimate it for lower-performing pages. By adding a correction step based on true rank distribution, the model better aligns its predictions across all rank ranges, ensuring more stable and realistic outputs.

Overall, these plots reveal a limitation of the model: while it is reliable for identifying top results, it tends to group lower-ranked items too closely together. This could reduce its usefulness in cases where distinguishing between less relevant results is important. (See Technical Annex 11.5 for more detail).

6.3.2.4.4 Fourth Model Evaluation

	Model	RMSE	MAE	Spearman	NDCG@10	CTR-Weighted@10	RMSE_1–3	RMSE_4–6	RMSE_7–10	RMSE_11–20	RMSE_21+
0	Random Forest	725.8789	22.4574	0.4634	0.1938	2.3003	1259.9988	587.0146	364.6142	532.7218	957.1372
1	XGBoost Custom Loss	819.2133	22.0847	0.4469	0.0226	1.8262	500.7832	979.4857	174.4650	596.6075	1177.0495
2	XGBoost Tuned	972.4780	23.3823	0.4595	0.0791	2.4689	313.3950	161.0137	60.8097	110.0880	1943.2537
3	XGBoost Tuned + Rank-Based Correction	964.4424	23.1525	0.4595	0.0791	2.4689	285.3602	134.0520	50.9544	174.1058	1821.0252

Table 4. Fourth Model Evaluation

Table 4 compares four models on their ability to predict search result rankings. While Random Forest achieves the best overall accuracy by having the lowest RMSE and strong ranking correlation, it performs poorly in the top-ranked positions (ranks 1–3), where precision is often most critical. In contrast, XGBoost with a custom loss excels at the top ranks but lacks consistency across others. The tuned XGBoost model significantly improves accuracy in mid-range ranks (4–20), though it performs worst in the lower-priority range (21+). Adding rank-based correction slightly improves its top-rank accuracy but still struggles in the tail. Overall, the

corrected XGBoost model provides the best accuracy at the top position given that Random Forest has balanced but less targeted performance.

6.3.2.5 Fifth Model: Neural Network

The inclusion of the custom-built neural network model in this experiment served two main purposes:

1. Benchmarking against more interpretable or traditional models like Random Forest and XGBoost.
2. Exploring nonlinear modeling capabilities on the dataset.

The relevance of incorporating a neural network within this modeling framework lies in its potential to model complex, non-linear feature interactions that may not be adequately captured by traditional tree-based models such as Random Forest or XGBoost. In particular, neural networks are well suited to exploit subtle hierarchical patterns in feature sets, which is valuable in ranking and personalized recommendation tasks. Moreover, a model such as RankNet, originally proposed for learning to rank tasks, aligns naturally with objectives like improving normalized discounted cumulative gain (NDCG) and Spearman correlation, which are essential for ranking quality assessment.

🔍 First 10 predictions for Neural Network:

	Real	Predicted	Error
0	1.65	1.00	-0.65
1	2.41	1.00	-1.41
2	2.24	1.00	-1.24
3	4.47	1.00	-3.47
4	3.73	1.00	-2.73
5	2.51	1.00	-1.51
6	1.91	5.76	+3.84
7	4.42	1.00	-3.42
8	3.68	1.00	-2.68
9	2.37	1.95	-0.42

Table 5. Fifth Model Prediction for Neural Network

The initial predictions, shown in table 5, made by the neural network model reveal a distinct pattern of underestimation. In 9 out of the first 10 cases, the model consistently predicts a value close to or exactly 1.00, regardless of the actual target, which results in a series of negative errors. This indicates a tendency of the model to default to a narrow prediction range, potentially due to underfitting or insufficient model complexity to capture underlying patterns in the data. The one exception (index 6), where the model significantly overestimates the value (5.76 vs. 1.91), stands out as an anomaly.

Overall, this behaviour suggests that the neural network, in its current form, lacks the flexibility or calibration to make fine-grained predictions, and may require further tuning, additional training data, or architectural changes to improve its generalization and predictive accuracy. While this may look like good predictions, we can see that our model just underpredicts to the best possible score for almost every result.

6.3.2.5.1 Fifth Model Evaluation

Model	RMSE	MAE	Spearman	NDCG@10	CTR-Weighted@10	RMSE_1-3	RMSE_4-6	RMSE_7-10	RMSE_11-20	RMSE_21+
Random Forest	725.8789	22.4574	0.4634	0.1938	2.3003	1259.9988	587.0146	364.6142	532.7218	957.1372
XGBoost Custom Loss	819.2133	22.0847	0.4469	0.0226	1.8262	500.7832	979.4857	174.4650	596.6075	1177.0495
XGBoost Tuned	972.4780	23.3823	0.4595	0.0791	2.4689	313.3950	161.0137	60.8097	110.0880	1943.2537
XGBoost Tuned + Rank-Based Correction	964.4424	23.1525	0.4595	0.0791	2.4689	285.3602	134.0520	50.9544	174.1058	1821.0252
Neural Network	960.1397	24.1183	0.2283	0.0718	1.6444	671.4514	474.4986	394.4192	181.1717	1251.2561

Table 6. Fifth Model Evaluation

Table 6 shows the performance of a Neural Network model for predicting search result rankings. Overall, the model has a relatively high RMSE (960.14) and a low Spearman correlation (0.2283), indicating limited accuracy and weak ability to preserve correct rank order. While its performance in the 1–3 rank range (RMSE: 671.45) is better than some models like Random Forest, it still falls short compared to XGBoost-based models. The model performs moderately in mid-rank ranges (4–20), but its error increases significantly for lower-priority results (RMSE_21+ = 1251.26). Additionally, both its NDCG@10 (0.0718) and CTR-weighted@10 (1.6444) scores are relatively low, suggesting it's less effective at ranking the most relevant results at the top. Overall, the Neural Network shows potential in some areas but underperforms in key metrics that matter for ranking-focused applications.

6.4 Feature Importance

Understanding which features contribute most to a model's predictions is a valuable step in both validating the model and guiding future improvements. Feature importance analysis helps identify the variables the model relies on most, which not only supports interpretability but also highlights areas where business or content strategy can be adjusted. This is particularly useful for communicating model behaviour to non-technical stakeholders and for focusing future optimization efforts on the significant factors.

Feature importance was extracted from the XGBoost model to determine which input variables had the most influence on the predictions. The results aligned well with domain knowledge, helping build confidence in the model's behaviour. This also provided a clearer understanding of what drives click-through rate (CTR) and highlighted areas for potential content and technical improvement.

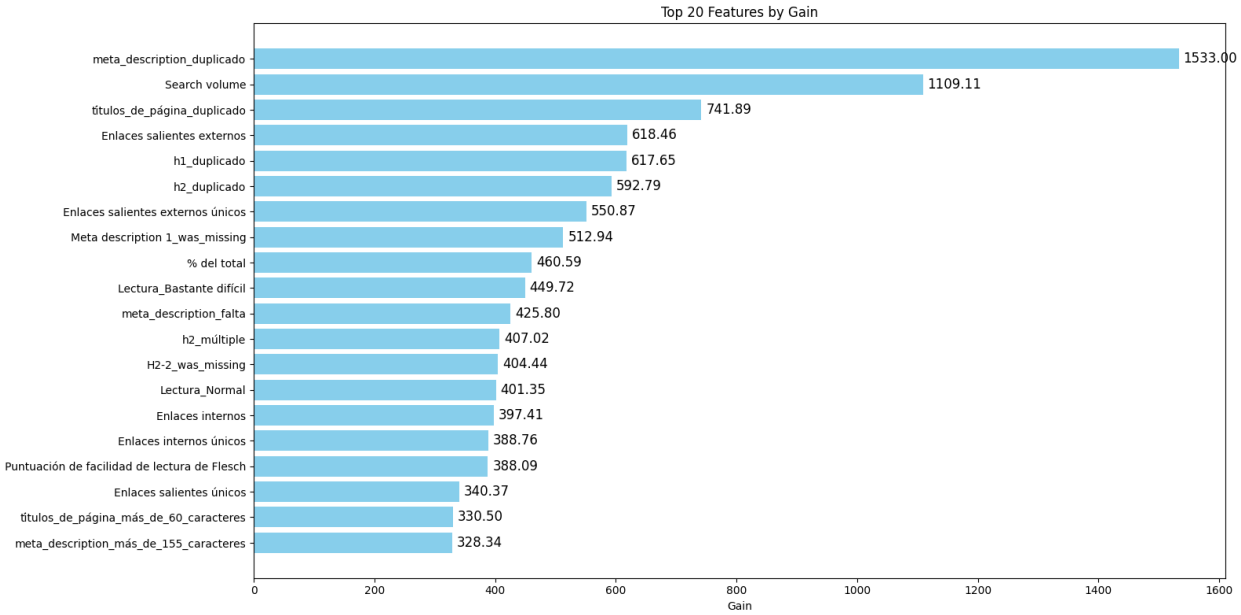


Figure 14. Top 20 Features by Gain

The top 20 features ranked by gain are shown in figure 14, which measures how much each feature contributes to reducing error in the model. The most impactful feature was the presence of duplicate meta descriptions, followed by search volume and duplicate page titles. These results emphasize how much metadata quality and search demand influence ranking predictions. Other key features included duplication of header tags, link structure which is both internal and external, and various readability measures such as Flesch reading ease scores and indicators of textual complexity. Features capturing missing elements such as meta description falta and Header 2 missing, also ranked high.

Altogether, the analysis highlights that both technical SEO factors and content quality signals are important in predicting CTR. These findings offer practical guidance: improving metadata uniqueness, ensuring complete page structure, and enhancing readability are likely to improve not just model performance but actual search visibility and user engagement.

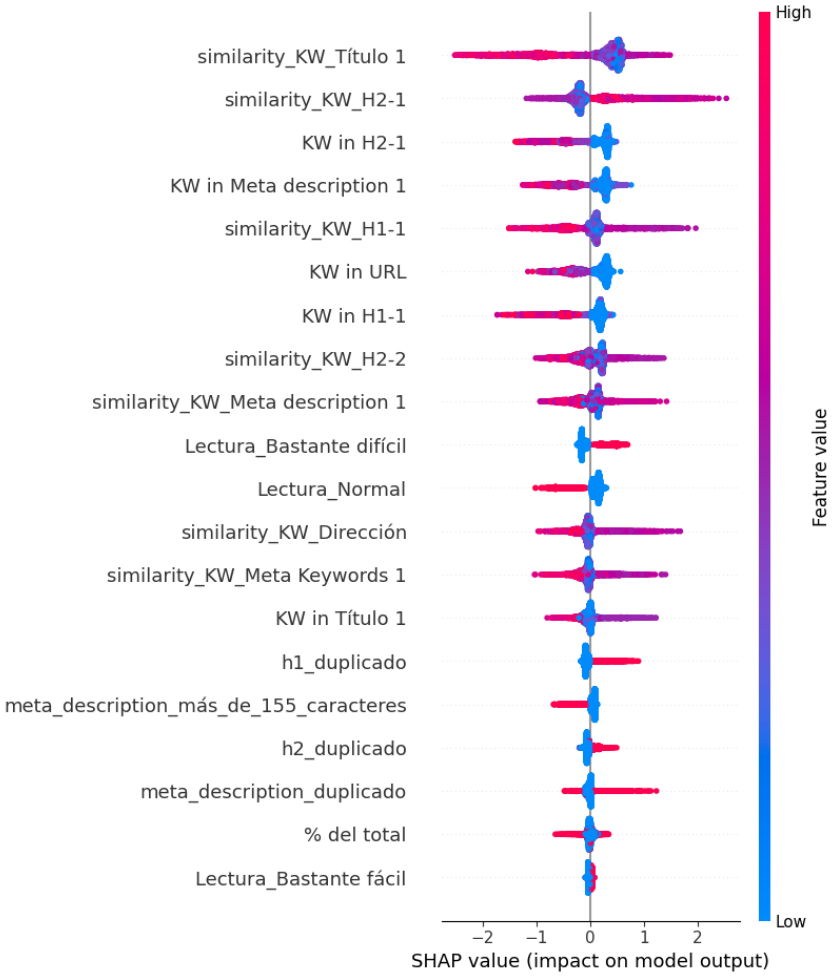


Figure 15. SHAP Value (impact on model output)

This SHAP plot, shown in figure 15, shows which features had the biggest influence on the model's predictions and in what direction. Features near the top had the most impact. The strongest influence came from how closely the main title matched the search keywords, followed by keyword similarity in H2 and H1 headings, and keyword presence in the meta description,

URL, and other key locations. These findings confirm that proper keyword placement across major on-page elements plays a critical role in predicting relevance.

The chart also shows that content readability affected predictions, difficult or normal reading levels had varying impacts, while very easy texts tended to lower the prediction slightly. Technical SEO issues, such as duplicated headings, long or missing meta descriptions, and overall content structure, also contributed, though to a lesser extent.

Overall, this analysis confirms that both keyword optimization and clear, well-structured content are important for improving the model's ranking predictions.

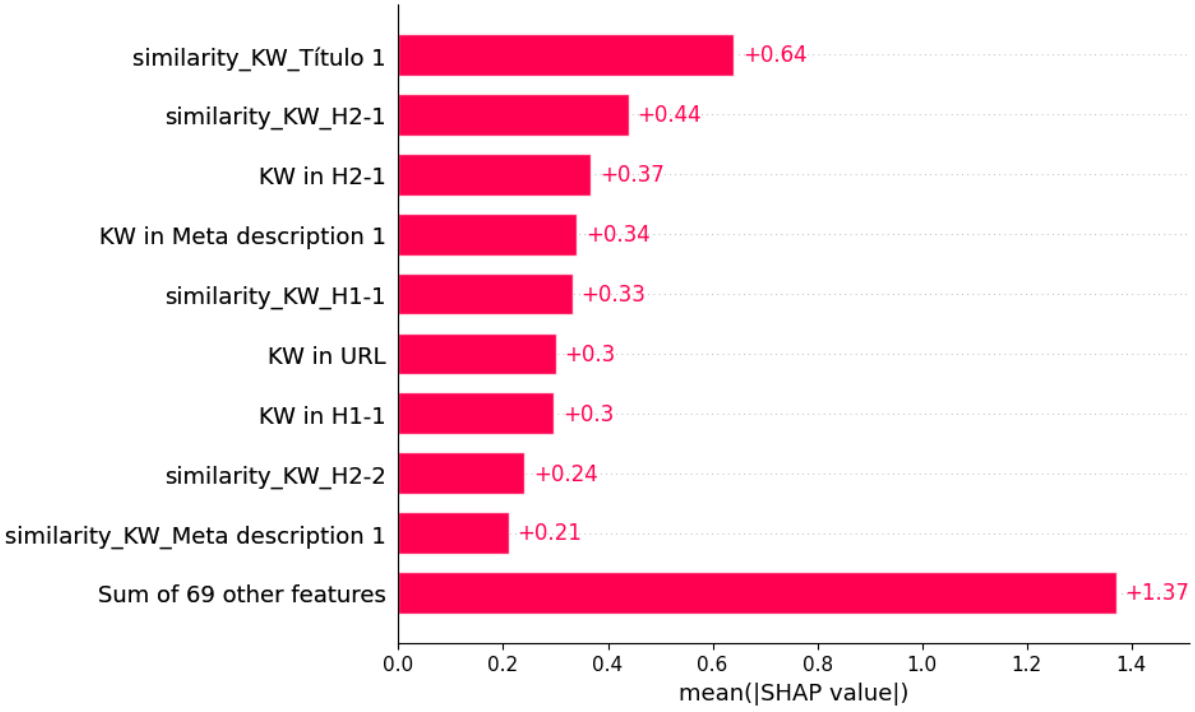


Figure 16. Highest Mean Absolute SHAP Values of the Model's Features

Figure 16 shows the top features that influenced the model's predictions, based on their average SHAP values. The most important factor was how closely the keywords in the main page title matched the search query, followed by keyword similarity in H2 headings and the presence of

keywords in those headings. Other key features included the appearance of keywords in the meta description, URL, and H1 tags. These results highlight the importance of placing relevant keywords in prominent page elements that search engines use to understand content. While these top features had strong individual impacts, the combined effect of many smaller features was even greater, showing that the model relies on a wide range of signals. Overall, this analysis reinforces the importance of both keyword placement and a balanced SEO strategy.

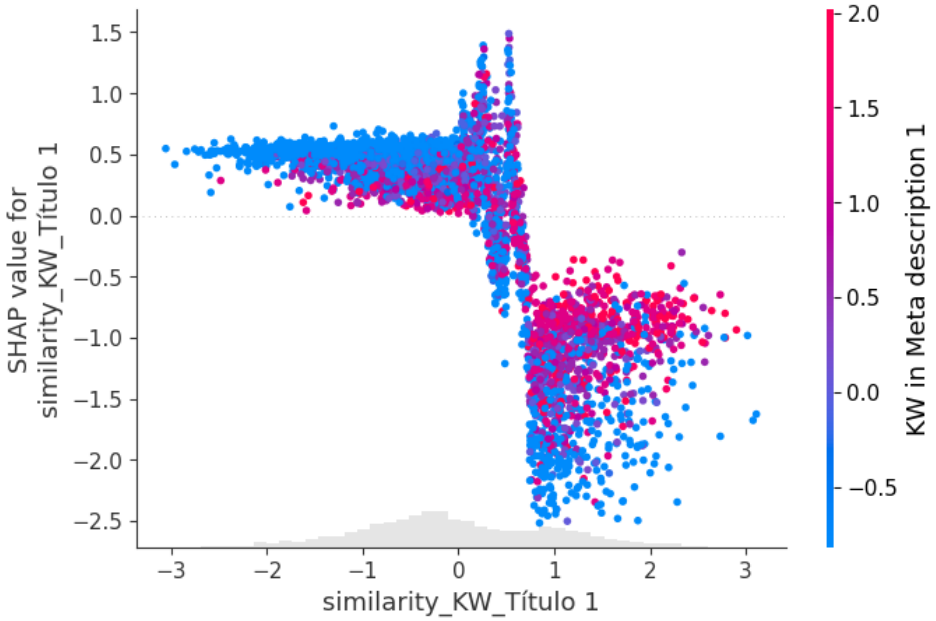


Figure 17. SHAP Dependence Scatter Plot of Keyword Similarity to the Page Title

The SHAP dependence plot shown in figure 17 shows that as the semantic similarity between a keyword and its given page title increases, the SHAP value decreases. As mentioned before, to find the similarity of the keyword to a given site feature, the cosine similarity of the keyword embedding and site feature embedding was taken. This meant that the tuned model predicted the page to have a lower (better) rank when the keyword and page title were more semantically similar. This improved rank is reinforced when the keyword also appears in the meta description, as shown by the large number of red instances at lower SHAP values.

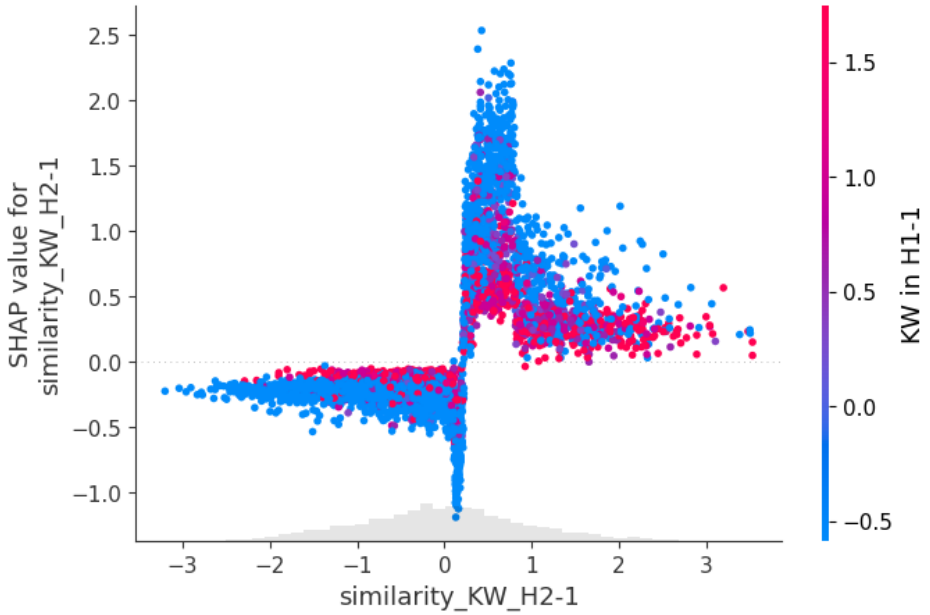


Figure 18. SHAP Dependence Scatter Plot of Keyword Similarity to H2 Header

In figure 18, the relationship between the keyword similarity to the H2 page header and the model's rank prediction. Interestingly this relationship goes against the expected trend and led the model to predict worse overall rankings for websites that have high keyword semantic similarity to the H2 header. Interestingly, if the keyword appeared in the H1 header, the negative effect of the keyword/H2 semantic similarity seemed to be mitigated. This result was not expected and will be discussed further in the key findings section.

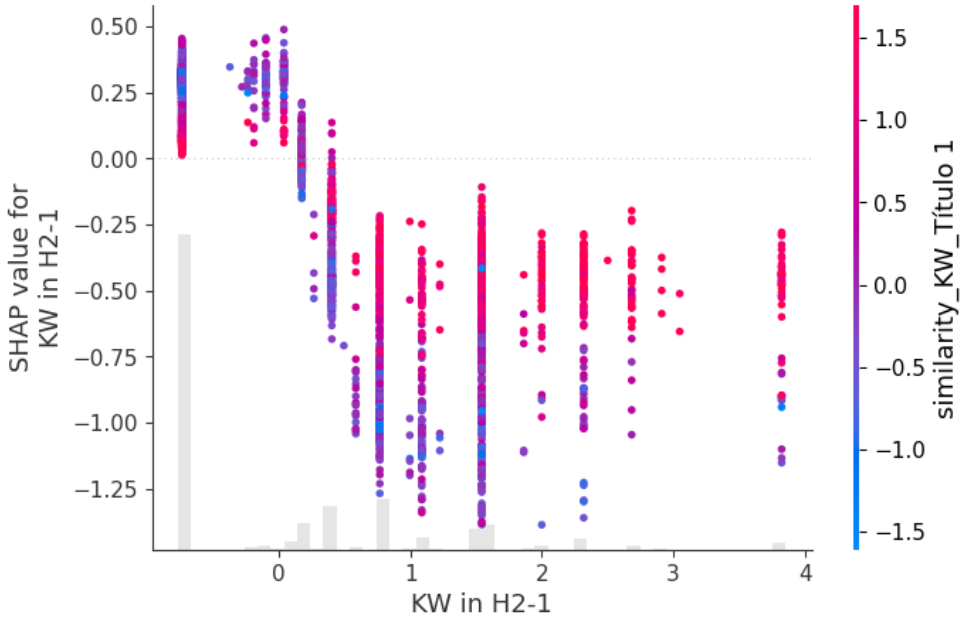


Figure 19. SHAP Dependence Scatter Plot of Keyword Presence in the H2 Header

The result from figure 18 is made more unexpected by figure 19, which shows that the presence (or partial presence) of the keyword in the H2 header generally improves the model's rank prediction. In this plot, the x-axis represents the percentage of the keyword that appears in the site feature, in this case the H2 header. This result aligned more with the expected result of better keyword optimised websites ranking higher in the Google Search. This result also linked to figure 17 and reinforced the idea that keyword alignment between page titles, headers and meta descriptions improves web page ranking.

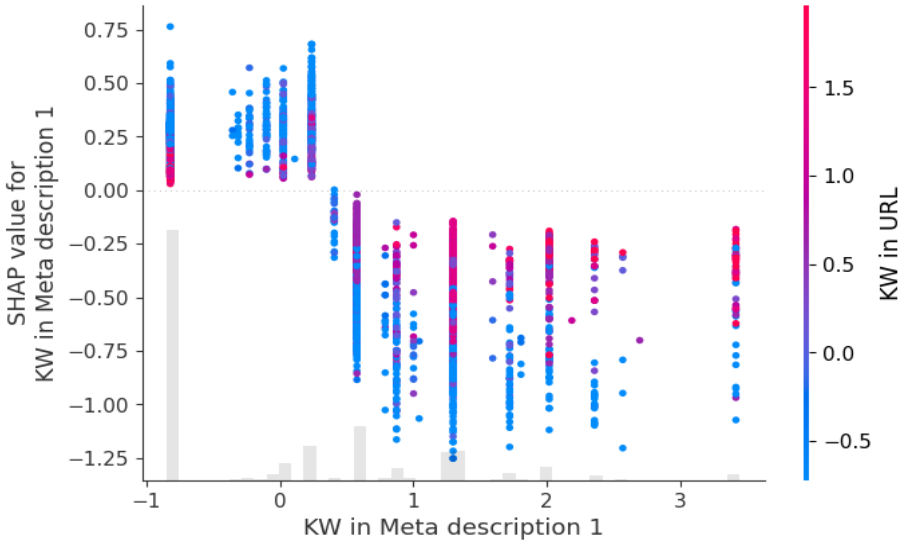


Figure 20. SHAP Dependence Scatter Plot of Keyword Presence in the Page Meta Description

This SHAP dependence plot shown in figure 20 examines the influence of keyword presence in the meta description. Similarly to what was shown in figure 17, the presence of the keyword in the meta description improved the model's rank prediction. If the keyword also appeared in the url, however, this appeared to have a reduced positive effect on the model's rank prediction, as shown by the collection of the red instances at less negative SHAP values.

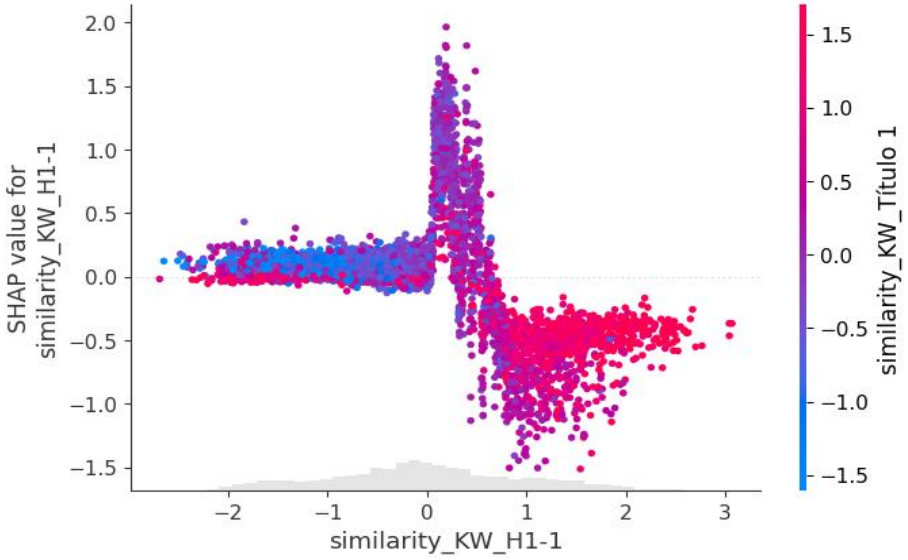


Figure 21. SHAP Dependence Scatter Plot of Keyword Similarity to H1 Header

The SHAP dependence plot shown in figure 21 shows that as the semantic similarity of the H1 header and keyword increase, the model's rank prediction improves. This effect is made stronger when the keyword is also semantically similar to the page title. (See Technical Annex 11.6 for more detail).

7. Key Findings and Strategy Recommendations

From these SHAP plots, there are some key takeaways that can be made. The first is that, as shown in figure 16 and the subsequent dependence plots, the semantic similarity and keyword presence have the most effect on a web pages ranking according to the model. This makes sense because keywords allow the web page to be tailored towards what users want and also provide a target for businesses to aim their content towards (Hermanson, 2021). It is, however, noteworthy that all of the top 9 features shown in figure 16 relate to the keyword, and could imply an issue with the keyword presence scoring or semantic embedding. As shown in figure 15, other features do have a large impact on the model's ranking decisions, such as difficulty of the google search crawler reading the web page, but these effects are still minor compared to the keyword. This should be investigated further to ensure the implied importance of the keyword is accurate (see future work).

From the SHAP dependence plots shown in figures 17-21, some key trends can be found that lead to future recommendations, as well as some interesting cases that are worth reinvestigating in more detail. Overall, it appears that having the keyword present in a given site feature or be systematically similar to that site feature, is good for the web pages search performance. This is especially true when the keyword is similar to or present in the page title, H1 header and meta description of the web page. However, as shown in figures 18 and 20, having the keyword be present in all site features could actually be detrimental to a site's performance. In these figures,

having the keyword be too similar to the H2 header or being present in the url as well as the meta description can harm the web pages performance in the search rankings. This result is further challenged by figure 19, which shows the web page has improved performance when the keyword appears in the H2 header. One possible explanation of this is the model has learnt that over-optimisation of a web page is actually bad for search ranking and is actively punished by the search algorithm (Mortensen, 2025). This trend certainly does exist in SEO and it is considered bad practice to stuff the web page with keywords (Mortensen, 2025). A web page that is over-optimised for keywords rather than overall user experience is actively punished by the algorithm and will harm the web pages ranking (Mortensen, 2025).

Therefore to summarise the meaning of these findings and the subsequent implications this should make to the strategy of La Magnética:

1. Ensure semantic similarity between keywords, page titles and H1 headers.
2. The impact of 1. can be compounded when the keyword is present in the meta description of the web page.
3. Keyword stuffing and over-optimisation should be avoided, especially in the H2 header and URL when other parts of the site are keyword optimised.

However, to say for definite if the tuned model created in this project has learnt this niche trend would be inaccurate without further analysis. This should be the basis of any future work performed using this model and the validity of this should be established before moving ahead with any further tasks.

8. Future Work

Following this project, there are several exciting opportunities that can be further explored, to truly maximise the potential of artificial intelligence in SEO. In this part of the report several of these opportunities will be discussed.

8.1 Validate Current Assertions

The first and most important next step is to validate the conclusions drawn from the SHAP analysis of the model. As previously mentioned, the model appeared to show that whilst keyword optimisation is good, over-optimisation can actually be detrimental to a websites ranking. This phenomenon is known as “keyword stuffing” and is considered bad practice in SEO (Mortensen, 2025). Google actively punishes websites that try and over-optimise rather than delivering the best results for the user (Mortensen, 2025). This is a very important factor to be aware of when optimising the site features such as the page title and headers. The fact that the model seemed to highlight shows a great ability for the model to understand the intricacies of SEO. However, at this stage it cannot be stated for definite that this was indeed what the model learned or if there was another issue in the data that happened to bring this SEO best practice to light. Further validation of this assertion will be needed to say for certain if the model has captured this knowledge. This could be done by taking a given instance and changing only the semantic similarity and keyword presence values. This could be done via a grid search, where each value combination is tested. The best rank performing combinations could then be recorded and this would confirm, not only if the previous assertions were accurate, but also indicate the best combination of keyword presence and semantic alignment in the site features to get the best ranking performance.

8.2 Develop Neural Network

While tree-based models such as XGBoost and Random Forest demonstrated strong predictive performance and interpretability, especially once they were tuned with custom loss functions to better capture the data, there is a strong potential to develop a fully integrated neural network that may be better at capturing complex trends and patterns. The neural network that was used in this project was very basic, with only a small number of linear layers and with the keywords and site features already embedded using sentence transformers. This severely underutilised the power of Neural Networks and Deep Learning, but a pragmatic approach had to be taken to ensure a strategy could be developed and delivered before the deadline.

With Neural Networks, we can combine text encoding and ranking prediction in one end-to-end architecture. Instead of relying on fixed similarity scores, we can feed raw textual data like keywords and page content directly into transformer-based layers (such as the SentenceTransformers used previously, or using token level encoders such as BERT). The network would then learn to generate embeddings that are optimized specifically for predicting ranking positions. This approach would allow the model to understand context more deeply and adjust its text representations based on actual SEO performance. By combining these text embeddings with other structured features like the CTR, search volume, or site technical health scores, it would be possible to capture more complex interactions and improve overall prediction accuracy.

To achieve this, it would be necessary to first redevelop the basic Neural Network to encode the keywords and site features within the transformer block of the network. More linear layers would also be needed to allow the network more opportunities to refine its predictions and minimize the loss function. Once a suitably complex model has been created, balancing the result performance with computational power, the hyperparameters should be tuned. Advanced regularization

techniques will be needed to avoid overfitting, and ensure the model generalises well on unseen data. Should this type of end-to-end neural model be successfully created, it would allow for a more adaptive and precise model that could deliver enhanced recommendations. This would then enable dynamic and complex SEO strategies to be developed that would help the clients of La Magnética perform ahead of their competitors in the rankings.

8.3 Performance Metric Analysis

During this project, the site performance was ignored from the dataset to avoid overcomplicating the feature data and making the model too complex to learn any meaningful insights. Whilst this was useful in allowing for keyword strategies to be found, this meant that certain aspects of the site such as response time or byte transfer were not taken into account in this model. These features would be worth investigating in another model to help further enrich the SEO strategies of La Magnética and allow for specific site performance kpis to be created. These would act as a target to optimise the site towards, ensuring from a performance aspect, the site was as well optimised as possible to reach the highest ranks.

8.4 Competitor Analysis

As a part of future development, it would be valuable to compare the SEO predictions from the La Magnética data and model with other industry competitors. This is an important set of features that was unable to be analysed in this project. Whilst the model created could predict the rank of a given url and keyword pair, it can only do this based on the other url-keyword pairs of the same site. By comparing our predictions to that of competitors we can better understand the model’s performance and practical effectiveness. However, gaining equivalent competitor data is very difficult as most of the data is well guarded with the companies. Tools such as Aherfs and SEMrush offer a solution to this problem and allow for at least some of the necessary data to be obtained from the competitors to be able to analyse and compare (Ahref, 2025; Semrush, 2025).

Both tools are used mostly for keyword tracking, site audits, backlink analysis and competitive intelligence. Whilst they have slight differences in the information they offer, combining the information they provide should provide sufficient competitor data to be able see how the model would predict ranks, not solely based on the clients of La Magnética but also their competitors. This will help to identify strengths and weaknesses of the sites managed by La Magnética and where a competitive advantage could be gained. Incorporating these insights will be crucial to strengthen the model's competitive positioning and to support broader adoption in real-world business contexts.

8.5 Scaling Framework

One of the most important focuses for the future should be scaling the AI model so it can handle larger and more diverse datasets. La Magnética manages more than one client and so as the model grows to support different types of websites, industries, and languages it must maintain both the performance and accuracy. To achieve this the framework needs to be optimised for modularity and scalability. This process can be broken down into six steps. The first is optimizing data pipelines. The datasets used in this project contained over 100,000 instances and this was only for one month of data on one specific company domain (VisitNavarra). The process of merging datasets to form the feature tables for model training will need to be optimised to handle even larger tables and made modular so the same infrastructure can be applied to different domains managed by La Magnética. The process of embedding the keywords and site features may need to be revisited, as this was already a very compute intensive process. This could be solved in two ways. The first relating to the previous future improvement suggested which was to train a complete Neural Network. This will allow the keywords and site features to be embedded during the model training and reduce the preprocessing time required. However, this will then cause the same computational time issues, just delayed to model training.

The other alternative relates to the second step of scaling the model which is to use distributed computing and use external compute resources. In this project, there was often the issue of hitting the Google Colab GPU runtime limit and this limited the model training that could be performed. Using external computing servers, such as AWS EC2 Accelerated Instances would allow for much more powerful models to be trained using much larger datasets. The enhanced model would also need more hyperparameter tuning to have better generalisation on different data and still maintain high performance levels. This would be the next step of scaling the framework created in this project and would mean strategies could be developed for different clients using the same model. This can be done by adding regularisation terms to the Neural Network, ensuring it is not overfitting to a given set of data. If the model is well generalised it will not need to be constantly retrained on data and so can be utilised more for making predictions and strategy recommendations.

An important step for scaling the model would be to enable multi-client support. This means designing the infrastructure such that it can securely handle and serve predictions for multiple clients together, ensuring data isolation and privacy. Whilst the clients of La Magnética are not in competing industries, it is still bad practice to allow data leaks within the model. These should be removed and the model framework edited in such a way that only authorised team members have access to the data of multiple clients. Relating to this any hard coded mentions of column names or features will need to be automated and redeployed to allow the model to work with multiple different client data. The second to last step of scaling the model will be to implement continuous integration and deployment (CI/CD). This will allow for the model to be automatically trained, improved, tested and employed without having to manually verify changes every time. This will speed up the workflow significantly and allow La Magnética to focus more on developing competitive strategies. This can be done using services like Git and GitHub which use GitHub Workflows to enable CI/CD.

Finally, building off of the continuous deployment, the addition of on demand API calls through Flexible API integration will be necessary to complete a fully scaled AI framework. This will require building scalable API endpoints that allow seamless integration into various business systems and marketing platforms. This will mean the models predictions and feature analysis will be available on demand and can be easily accessed by both La Magnética and their clients. By focusing on these improvements, the AI model will scale to be better positioned to serve larger enterprises, support international deployments, and deliver consistent value as demand and market grows.

9. Conclusion

The development and evaluation of multiple predictive models, including traditional machine learning approaches (Random Forest and XGBoost), Neural Networks, and a customized XGBoost model, have led to a framework that can be used to predict and rank where a web page will appear in Google Search. The model had to perform best at the lowest ranks, as this is where the most search traffic happens and so is most important to focus on SEO improvements. The XGBoost model with a custom loss function based on expected click-through rate (CTR) times search volume proved to be the best performing model. The integration of a residual correction linear regression model further enhanced the calibration of predictions, ensuring better alignment with observed values while avoiding overfitting. This helped to reduce the linear trend of underpredicting the performance at top ranks and overpredicting the performance at the bottom ranks.

Once a suitably accurate model had been established, the features could be analysed to help develop a strategy for La Magnética going forward. The feature importance and dependence using SHAP values would allow for accurate recommendations of what features of a website are

most important to optimize to improve the page's search ranking. From the SHAP analysis performed on the model, it was found that the presence and semantic similarity of site features to keywords had the most impact on improving or harming a web page's ranking. It is recommended that the main features that should be focused on when optimising a website are page titles and H1 headers. The keyword should then also be included in other features such as the meta description to further boost the keyword optimisation. However, keyword stuffing and over-optimisation should be avoided when other parts of the site are keyword optimised.

These recommendations are subject to change based on the future work recommended (see Future Work) and the further investigation required for the impact of keywords and H2 headers on model performance (see SHAP Analysis Key Findings and Future Work). After the results of this project have been validated there are several exciting ways this project can be developed further. From building a fully scaled and continuously integrated and deployed Neural Network to performing an in-depth competition analysis, the project has laid the groundwork for future projects to truly enhance the strategy of La Magnética and maximise the potential of Artificial Intelligence in Search Engine Optimisation.

10. Bibliography / References

Ahref. (2025). *Ahrefs—Marketing Intelligence Tools Powered by Big Data.* Ahrefs. https://ahrefs.com/

Awan, A. A. (2023, June 28). *An Introduction to SHAP Values and Machine Learning Interpretability*. https://www.datacamp.com/tutorial/introduction-to-shap-values-machine-learning-interpretability

Breiman, L. (2001). Random Forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* (No. arXiv:1810.04805). arXiv. https://doi.org/10.48550/arXiv.1810.04805

Enge, E., Spencer, S., & Stricchiola, J. (2022). *The Art of SEO, 4th Edition*. https://learning.oreilly.com/library/view/the-art-of/9781098102609/

Gupta, S. (2025, January 6). *Sentence Transformer*. https://www.geeksforgeeks.org/nlp/sentence-transformer/

Hermanson, T. (2021, March 14). *Why Keywords Are Still So Very Important for SEO*. https://www.searchenginejournal.com/seo/why-keywords-important-seo/

Mortensen, O. (2025, April 25). *Why is Over-optimizing bad for SEO? [How to Avoid in 2025]*. https://seo.ai/blog/why-is-over-optimizing-bad#:~:text=When%20search%20engines%20pick%20up%20on%20poor%20SEO,old%20SEO%20tricks%20have%20become%20even%20more%20dangerous.

Regulation—2016/679—EN - gdpr—EUR-Lex. (n.d.). Retrieved 7 July 2025, from https://eur-

lex.europa.eu/eli/reg/2016/679/oj/eng

Screaming Frog SEO Spider Website Crawler. (n.d.). *Screaming Frog*. Retrieved 7 July 2025, from https://www.screamingfrog.co.uk/seo-spider/

Search Console Help. (n.d.). Retrieved 7 July 2025, from https://support.google.com/webmasters/#topic=9128571

Semrush. (2025). *SEO Competitive Analysis Tools*. https://www.semrush.com/features/seo-competitor-analysis-tool/

11. Technical Annex

This contains the most relevant sections of code from the project for the data transformation and feature engineering, model training and SHAP analysis. The full code, schemas and tables used will be uploaded separately in the Technical Annex .zip file.

11.1 Sentence Transformers

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd
import numpy as np
import os
from more_itertools import chunked
from datetime import datetime
import gc

# Load model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Define columns
keyword_col = 'KW'
site_feature_cols = ['Dirección', 'Título 1', 'Meta description 1', 'Meta Keywords 1', 'H1-1', 'H2-1', 'H2-2']
embedding_dim = 384
batch_size = 512

# Timestamped folder in Google Drive
timestamp = datetime.now().strftime('%Y%m%d_%H%M')
drive_base = '/content/drive/MyDrive/Machine Learning/embedding_outputs'
embedding_dir = os.path.join(drive_base, f'embeddings_{timestamp}')
features_path = os.path.join(drive_base, f'df_with_similarity_{timestamp}.parquet')

# Create folders
os.makedirs(embedding_dir, exist_ok=True)

# Helper: batch encode and write to disk
def encode_and_save_column(col_name, texts, output_path):
    with open(output_path, 'wb') as f:
        for chunk in chunked(texts, batch_size):
            emb = model.encode(chunk, convert_to_numpy=True)
            emb.astype(np.float32).tofile(f)

# Step 1: Encode and save keyword embeddings
kw_texts = df[keyword_col].astype(str).tolist()
kw_path = os.path.join(embedding_dir, f'{keyword_col}.bin')
encode_and_save_column(keyword_col, kw_texts, kw_path)
```

```
# Step 2: Process each site feature one at a time
for col in site_feature_cols:
    print(f'Processing: {col}')
    texts = df[col].astype(str).tolist()
    col_path = os.path.join(embedding_dir, f'{col}.bin')

    encode_and_save_column(col, texts, col_path)

    # Load embeddings in chunks to compute similarity
    sim_scores = []
    with open(kw_path, 'rb') as f_kw, open(col_path, 'rb') as f_col:
        while True:
            kw_chunk = np.fromfile(f_kw, dtype=np.float32, count=batch_size * embedding_dim)
            col_chunk = np.fromfile(f_col, dtype=np.float32, count=batch_size * embedding_dim)
            if kw_chunk.size == 0 or col_chunk.size == 0:
                break
            kw_chunk = kw_chunk.reshape(-1, embedding_dim)
            col_chunk = col_chunk.reshape(-1, embedding_dim)
            sim = cosine_similarity(kw_chunk, col_chunk).diagonal()
            sim_scores.extend(sim)

    df[f'similarity_KW_{col}'] = sim_scores
    gc.collect()

# Step 3: Fix column type for Parquet compatibility
df[keyword_col] = df[keyword_col].astype(str)

# Step 4: Save final DataFrame to Google Drive
df.to_parquet(features_path, index=False)

print(f"All files saved to Google Drive at:\n→ {drive_base}")
```

11.2 Evaluation Function

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_squared_error, mean_absolute_error, ndcg_score
from scipy.stats import spearmanr
from datetime import datetime
import os
from IPython.display import display

# CTR Decay Parameters
a_opt = 1.0
b_opt = 4.021
c_opt = 2.7671

# Custom CTR Curve Evaluation Metric
def ctr_weight_np(position_array):
    base = np.clip(position_array + b_opt, a_min=1e-3, a_max=None)
    return a_opt / np.power(base, c_opt)

def ctr_weighted_score(y_true, y_pred, k=10):
    top_k_indices = np.argsort(-y_pred)[:k]
    relevance = y_true[top_k_indices]
    positions = np.arange(1, k + 1)
    ctr_weights = ctr_weight_np(positions)
    return (relevance * ctr_weights).sum()

def evaluate_models(results: dict, y_test: np.ndarray, k: int = 10):
    y_test = np.asarray(y_test)
    timestamp = datetime.now().strftime('%Y%m%d_%H%M')

    # Custom Google Drive Path
    base_dir = '/content/drive/MyDrive/Machine Learning/Model Evaluation'
    #base_dir = '/content/drive/MyDrive/Trimester 3/Final Project/Model Evaluation'
    save_path = os.path.join(base_dir, f'eval_{timestamp}')
    os.makedirs(save_path, exist_ok=True)

    metrics = []
    rmse_bin_plot_data = []
```

```python
for name, preds in results.items():
    preds = np.asarray(preds)
    rmse = mean_squared_error(y_test, preds)
    mae = mean_absolute_error(y_test, preds)
    spearman_corr, _ = spearmanr(y_test, preds)

    true_rel = (1 / (y_test + 1e-5)).reshape(1, -1)
    pred_scores = preds.reshape(1, -1)
    ndcg = ndcg_score(true_rel, pred_scores, k=k)
    ctr_score = ctr_weighted_score(y_test, preds, k=k)

    # Sample every 50th point
    sample_idx = np.arange(0, len(y_test), 50)
    y_sample = y_test[sample_idx]
    p_sample = preds[sample_idx]
    residuals = p_sample - y_sample

    # RMSE by rank bin
    bin_labels = ['1-3', '4-6', '7-10', '11-20', '21+']
    rank_bins = pd.cut(y_sample, bins=[0, 3, 6, 10, 20, np.inf], labels=bin_labels)
    rmse_by_bin = {}

    for label in bin_labels:
        bin_mask = rank_bins == label
        if bin_mask.sum() > 0:
            bin_rmse = mean_squared_error(y_sample[bin_mask], p_sample[bin_mask])
            rmse_by_bin[f'RMSE_{label}'] = round(bin_rmse, 4)
            rmse_bin_plot_data.append({'Model': name, 'Rank Bin': label, 'RMSE': bin_rmse})
        else:
            rmse_by_bin[f'RMSE_{label}'] = np.nan

    metrics.append({
        'Model': name,
        'RMSE': rmse,
        'MAE': mae,
        'Spearman': spearman_corr,
        f'NDCG@{k}': ndcg,
        f'CTR-Weighted@{k}': ctr_score,
        **rmse_by_bin
    })
```

```python
# 1. True vs. Predicted Ranks
plt.figure(figsize=(6, 4))
plt.plot(y_sample, marker='o', label='True Rank', alpha=0.8)
plt.plot(p_sample, marker='x', label='Predicted Rank', alpha=0.8)
plt.title(f'{name} — True vs. Predicted Ranks')
plt.xlabel('Sample Index')
plt.ylabel('Rank (lower is better)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(save_path, f'{name}_lineplot.png'))
plt.close()

# 2. Residuals vs. True Rank
plt.figure(figsize=(6, 4))
plt.scatter(y_sample, residuals, alpha=0.4)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('True Rank')
plt.ylabel('Residual (Pred - True)')
plt.title(f'{name} — Residuals vs. True Rank')
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(save_path, f'{name}_residual_scatter.png'))
plt.close()

# 3. Residuals by Rank Range (Boxplot)
plt.figure(figsize=(6, 4))
sns.boxplot(x=rank_bins, y=residuals)
plt.title(f'{name} — Residuals by True Rank Range')
plt.xlabel('True Rank Range')
plt.ylabel('Residual')
plt.tight_layout()
plt.savefig(os.path.join(save_path, f'{name}_residual_boxplot.png'))
plt.close()
```

```
    # Save Real vs. Predicted vs. Error table
    comparison_df = pd.DataFrame({
        'Real': y_test,
        'Predicted': preds,
        'Error': preds - y_test
    })
    comparison_df.to_csv(os.path.join(save_path, f'{name}_comparison.csv'), index=False)

    # Display styled table
    styled_table = (
        comparison_df.head(10)
        .style.background_gradient(cmap='coolwarm', subset=['Error'])
        .format({'Real': '{:.2f}', 'Predicted': '{:.2f}', 'Error': '{:+.2f}'})
    )
    print(f"\nFirst 10 predictions for {name}:")
    display(styled_table)

# Save metrics
metrics_df = pd.DataFrame(metrics).round(4)
metrics_df.to_csv(os.path.join(save_path, 'model_metrics.csv'), index=False)

# RMSE by Rank Bin Plot
rmse_bin_df = pd.DataFrame(rmse_bin_plot_data)
plt.figure(figsize=(8, 5))
sns.barplot(data=rmse_bin_df, x='Rank Bin', y='RMSE', hue='Model')
plt.title('RMSE by Rank Bin for Each Model')
plt.ylabel('RMSE')
plt.tight_layout()
plt.savefig(os.path.join(save_path, 'rmse_by_rank_bin.png'))
plt.close()

print(f"\nEvaluation complete. Results saved to:\n→ {save_path}")
return metrics_df
```

11.3 Custom Loss Function for XGBoost

```
import numpy as np
import pandas as pd
import xgboost as xgb
from sklearn.ensemble import RandomForestRegressor

# CTR decay parameters
a_opt = 1.0
b_opt = 4.021
c_opt = 2.7671
alpha = 0.9  # Balance between weighted and unweighted loss

# CTR weight function
def ctr_weight(position):
    return a_opt / (position + b_opt) ** c_opt

# Top-rank penalty function (higher for lower ranks)
def top_rank_penalty(labels):
    return 1 / np.log1p(labels)  # much steeper than exp(-labels)

def custom_weighted_mse_with_contrast(preds, dtrain):
    labels = dtrain.get_label().astype(np.float32)
    ctr_weights = ctr_weight(labels)
    search_volume = dtrain.get_weight()
    penalty = top_rank_penalty(labels)

    combined_weights = ctr_weights * search_volume * penalty

    errors = preds - labels

    # Main gradient and hessian (weighted MSE)
    grad = 2 * (alpha * combined_weights * errors + (1 - alpha) * errors)
    hess = 2 * (alpha * combined_weights + (1 - alpha))

    # Contrastive penalty: amplify gradient for top-ranked items
    contrast_weight = np.where(labels <= 10, 5.0, 1.0)
    grad *= contrast_weight
    hess *= contrast_weight

    return grad, hess

def custom_eval(preds, dtrain):
    labels = dtrain.get_label().astype(np.float32)
    ctr_weights = ctr_weight(labels)
    search_volume = dtrain.get_weight()
    penalty = top_rank_penalty(labels)

    combined_weights = ctr_weights * search_volume * penalty

    errors = preds - labels
    loss = ((alpha * combined_weights + (1 - alpha)) * (errors**2)).mean()
    return 'cw_mse', float(loss)
```

11.4 XGBoost Custom Loss Tuning

```
from sklearn.model_selection import ParameterSampler
from scipy.stats import uniform, randint
import random

# Define parameter space
param_dist = {
    'max_depth': randint(4, 14),
    'eta': uniform(0.01, 0.4),
    'subsample': uniform(0.4, 0.6),
    'colsample_bytree': uniform(0.2, 0.8),
    'lambda': uniform(0, 20),
    'alpha': uniform(0, 20)
}

# Sample 20 random combinations
n_iter = 20
param_list = list(ParameterSampler(param_dist, n_iter=n_iter, random_state=42))

best_score = float('inf')
best_model = None
best_params = None

print("Starting hyperparameter tuning...")

for i, params in enumerate(param_list):
    print(f"\nTrial {i+1}/{n_iter} — Params: {params}")

    full_params = {
        'tree_method': 'hist',
        'device': 'cuda',
        'seed': 42,
        'max_depth': params['max_depth'],
        'eta': params['eta'],
        'subsample': params['subsample'],
        'colsample_bytree': params['colsample_bytree'],
        'lambda': params['lambda'],
        'alpha': params['alpha']
    }
```

```
    model = xgb.train(
        params=full_params,
        dtrain=dtrain,
        num_boost_round=200,
        evals=[(dtest, 'eval')],
        early_stopping_rounds=20,
        obj=custom_weighted_mse_with_contrast,
        feval=custom_eval,
        verbose_eval=False
    )

    preds = np.clip(model.predict(dtest), a_min=1.0, a_max=None)
    #score_name, score_value = custom_eval(preds, dtest)
    score_name, score_value = rmse_top3(preds, dtest)

    print(f"{score_name}: {score_value:.4f}")

    if score_value < best_score:
        best_score = score_value
        best_model = model
        best_params = params

print("\nTuning complete.")
print(f"Best RSME 1-3: {best_score:.4f}")
print(f"Best Parameters: {best_params}")
```

11.5 Linear Regression Rank Correction

```
from sklearn.preprocessing import StandardScaler

# Fit on training data, transform both train and test
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
import xgboost as xgb

model = xgb.Booster()
model.load_model("/content/drive/MyDrive/Machine Learning/best_xgb_ctr_model.json")
#model.load_model("/content/drive/MyDrive/Trimester 3/Final Project/best_xgb_ctr_model.json")
```

```
# Assuming you already have predictions from your best model
dtrain = xgb.DMatrix(X_train)
y_pred_train = xgb_model.predict(dtrain)
```

```
from sklearn.linear_model import LinearRegression
import joblib

# Use true rank as the only feature
rank_feature_train = y_train.values.reshape(-1, 1)
residuals_train = y_pred_train - y_train

# Fit correction model
rank_correction = LinearRegression()
rank_correction.fit(rank_feature_train, residuals_train)

# Predict and correct
residual_preds = rank_correction.predict(rank_feature_train)
corrected_preds = y_pred_train - residual_preds

# Clip predictions to ensure they are ≥ 1
corrected_preds = np.clip(corrected_preds, a_min=1.0, a_max=None)

# Save the correction model
joblib.dump(rank_correction, "/content/drive/MyDrive/Machine Learning/rank_residual_correction.pkl")
#joblib.dump(rank_correction, "/content/drive/MyDrive/Trimester 3/Final Project/rank_residual_correction.pkl")
```

```python
# Validate Residual Correction To Prevent Overfitting
from sklearn.model_selection import train_test_split

# Split test set into validation and holdout folds
X_val, X_holdout, y_val, y_holdout, y_pred_val, y_pred_holdout = train_test_split(
    X_test, y_test, results['XGBoost Tuned'], test_size=0.5, random_state=42
)
```

```python
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

# Use true rank as the only feature for correction
rank_val = y_val.values.reshape(-1, 1)
residuals_val = y_pred_val - y_val

# Fit correction model
correction_model = LinearRegression()
correction_model.fit(rank_val, residuals_val)
```

▾ LinearRegression ❶ ❷

```
LinearRegression()
```

```python
# Predict residuals on holdout
rank_holdout = y_holdout.values.reshape(-1, 1)
residual_preds_holdout = correction_model.predict(rank_holdout)

# Correct predictions
corrected_preds_holdout = y_pred_holdout - residual_preds_holdout
```

```python
from sklearn.metrics import mean_squared_error, mean_absolute_error
from scipy.stats import spearmanr

# Before correction
rmse_before = mean_squared_error(y_holdout, y_pred_holdout)
mae_before = mean_absolute_error(y_holdout, y_pred_holdout)
spearman_before = spearmanr(y_holdout, y_pred_holdout).correlation

# After correction
rmse_after = mean_squared_error(y_holdout, corrected_preds_holdout)
mae_after = mean_absolute_error(y_holdout, corrected_preds_holdout)
spearman_after = spearmanr(y_holdout, corrected_preds_holdout).correlation

print("Holdout Evaluation:")
print(f"RMSE Before: {rmse_before:.2f} → After: {rmse_after:.2f}")
print(f"MAE  Before: {mae_before:.2f} → After: {mae_after:.2f}")
print(f"Spearman Before: {spearman_before:.4f} → After: {spearman_after:.4f}")
```

```python
# Overall Model Predictor Class
import xgboost as xgb
import joblib
import numpy as np

class RankPredictor:
    def __init__(self, xgb_model_path: str, correction_model_path: str):
        # Load XGBoost model
        self.xgb_model = xgb.Booster()
        self.xgb_model.load_model(xgb_model_path)

        # Load residual correction model
        self.correction_model = joblib.load(correction_model_path)

    def predict(self, X: np.ndarray, clip_min: float = 1.0) -> np.ndarray:
        """
        Predict corrected ranks for new data.

        Parameters:
        - X: Feature matrix (NumPy array or DataFrame)
        - clip_min: Minimum allowed rank (default = 1.0)

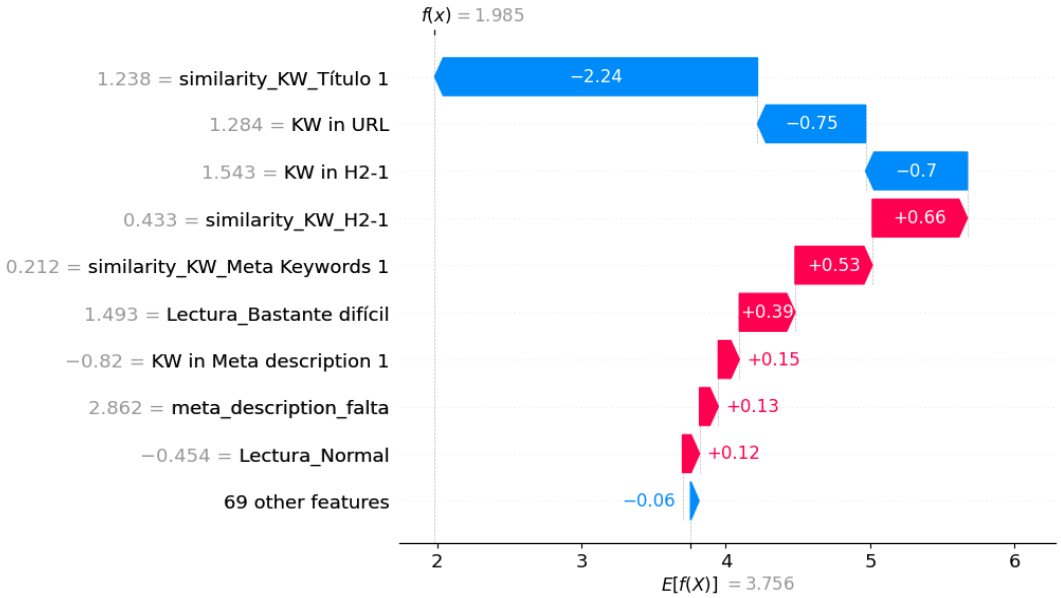
        Returns:
        - Corrected rank predictions (NumPy array)
        """
        # Predict with XGBoost
        dmatrix = xgb.DMatrix(X)
        y_pred = self.xgb_model.predict(dmatrix)

        # Predict residuals using predicted rank as proxy
        rank_proxy = y_pred.reshape(-1, 1)
        residual_correction = self.correction_model.predict(rank_proxy)

        # Apply correction and clip
        y_corrected = np.clip(y_pred - residual_correction, a_min=clip_min, a_max=None)
        return y_corrected
```

11.6 SHAP Analysis Values

11.6.1 How each feature changes the rank from the baseline rank



11.6.2 How the sum of the SHAP values across all features changes the rank of a given instance

	Instance	Base Value	Sum of SHAP Values	Final Prediction (Rank)
0	0	3.756167	-1.771391	1.984776
1	1	3.756167	1.896885	5.653052
2	2	3.756167	-2.756167	1.000000
3	3	3.756167	-2.756167	1.000000
4	4	3.756167	-0.472400	3.283768
5	5	3.756167	-1.430035	2.326132
6	6	3.756167	0.223646	3.979813
7	7	3.756167	-2.297951	1.458216
8	8	3.756167	1.896885	5.653052
9	9	3.756167	-2.756167	1.000000

11.6.3 SHAP Value Code

```
import xgboost as xgb
import joblib
import shap
import numpy as np
import pandas as pd

# ✅ Paths to your saved models
xgb_model_path = "/content/drive/MyDrive/Machine Learning/best_xgb_ctr_model.json"
correction_model_path = "/content/drive/MyDrive/Machine Learning/rank_residual_correction.pkl"

#xgb_model_path = "/content/drive/MyDrive/Trimester 3/Final Project/best_xgb_ctr_model.json"
#correction_model_path = "/content/drive/MyDrive/Trimester 3/Final Project/rank_residual_correction.pkl"

# Instantiate your predictor
predictor = RankPredictor(xgb_model_path, correction_model_path)

# Wrap the predictor for SHAP
def corrected_predict(X):
    return predictor.predict(X)

# Rewrap the scaled array as a DataFrame with the original column names
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X_test.columns)

# Create SHAP explainer
explainer = shap.Explainer(corrected_predict, X_test_scaled_df)

# Compute SHAP values
shap_values = explainer(X_test_scaled_df)

# Generate SHAP summary plot
shap.summary_plot(shap_values, X_test_scaled_df, feature_names=X.columns)
```